

EXAMEN SESSION NORMALE SEMESTRE 2

Année Académique : 2023/2024 – INF 332- Ingénierie des applications web – IN3 Durée : 3H

Exercice 1 (4,5 pts)

- 1) Définir les termes suivants : JSP, Bean . (1 pt)
- 2) Qu'est-ce qu'une session en Java EE et pourquoi est-elle importante ? (1 pt)
- 3) Expliquer comment créer, lire et supprimer un attribut de session en Java EE. Fournir un exemple de code pour chaque opération. ? (1,5pts)
- 4) Donner la Structure d'une servlet HTTP (1 pt)

Exercice 2 (4,5 pts)

- 1) Décrire le rôle des cookies en Java EE et comment ils diffèrent des sessions. ? (1,5 pts)
- 2) Écrire un exemple de code pour créer, lire et supprimer un cookie en Java EE (1,5 pts)
- 3) Définir le terme portée dans une application, Citez 4 types de portées (1,5 pts)

Problème (11 pts)

Contexte: Vous travaillez sur un projet de gestion des dossiers médicaux électroniques (DME) pour un hôpital. Vous devez créer un module applicatif pour la création, la mise à jour, la suppression et la visualisation des dossiers médicaux des patients. Les fonctionnalités attendues sont les suivantes: Création d'un dossier médical (creerDossier.jsp): Permet de créer un dossier médical en saisissant des données depuis un formulaire. Mise à jour d'un dossier médical (mettreAJourDossier.jsp): Permet de mettre à jour un dossier médical existant. Suppression d'un dossier médical (supprimerDossier.jsp): Permet de supprimer un dossier médical en saisissant l'identifiant du patient. Affichage du dossier médical (afficheDossier.jsp): Permet de visualiser la fiche du dossier médical créé.

Attributs d'un dossier médical: identifiant patient, nom, prénom, date de naissance, adresse, numéro de téléphone, historique médical, traitements en cours, allergies, médecin référent.

1. Proposer le contenu du fichier web.xml pour effectuer le mapping pour nos différentes fonctionnalités. (1,5 pts)
2. Créer le bean DossierMedical. (2 pts)
3. Créer le formulaire creerDossier.jsp qui redirige vers afficheDossier.jsp. (2 pts)
4. Proposer un code pour le contrôleur CreationDossier pour gérer les fonctionnalités des dossiers médicaux. (2 pts)
5. Créer la page afficheDossier.jsp. (2 pts)
6. Créer le formulaire mettreAJourDossier.jsp pour la mise à jour des dossiers médicaux. (2 pts)
7. Créer le formulaire supprimerDossier.jsp pour la suppression des dossiers médicaux. (2 pts)
8. Proposer un code pour le contrôleur SuppressionDossier pour gérer la suppression des dossiers médicaux. (2 pts)

CORRECTION EXAMEN SESSION NORMALE SEMESTRE 2

Année Académique : 2023/2024 – INF 332- Ingénierie des applications web – IN3 Durée : 3H
Exercice 1 :

1. JSP (JavaServer Pages) est une technologie Java qui permet de générer dynamiquement du contenu web. Un Bean est un composant Java réutilisable qui encapsule des données et des méthodes.
2. Une session en Java EE est un mécanisme qui permet de stocker des informations relatives à un utilisateur entre plusieurs requêtes HTTP. Elle est importante car elle permet de maintenir l'état entre les différentes pages d'une application web.
3. Pour créer un attribut de session, on utilise HttpSession.setAttribute(). Pour le récupérer, on utilise HttpSession.getAttribute(). Pour le supprimer, on utilise HttpSession.removeAttribute().
Exemple :

```
java
// Créer un attribut de session
HttpSession session = request.getSession();
session.setAttribute("nomAttribut", "valeurAttribut");

// Récupérer un attribut de session
String valeur = (String) session.getAttribute("nomAttribut");

// Supprimer un attribut de session
session.removeAttribute("nomAttribut");
```

4. La structure d'une servlet HTTP est la suivante :

```
java
public class MaServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Logique à exécuter lors d'une requête GET
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Logique à exécuter lors d'une requête POST
    }
}
```

Exercice 2 :

1. Les cookies en Java EE sont de petits fichiers stockés sur le poste client, contrairement aux sessions qui sont stockées côté serveur. Ils permettent de stocker des informations sur l'utilisateur entre les différentes requêtes.
2. Exemple de code pour créer, lire et supprimer un cookie :

```
java
// Création d'un cookie
```

```

Cookie cookie = new Cookie("nomCookie", "valeurCookie");
cookie.setMaxAge(3600); // Expiration dans 1 heure
response.addCookie(cookie);

// Lecture d'un cookie
Cookie[] cookies = request.getCookies();
for (Cookie c : cookies) {
    if (c.getName().equals("nomCookie")) {
        String valeur = c.getValue();
        // Faire quelque chose avec la valeur
    }
}

// Suppression d'un cookie
Cookie cookie = new Cookie("nomCookie", "");
cookie.setMaxAge(0);
response.addCookie(cookie);

```

3. La portée dans une application Java EE fait référence à la durée de vie et à la visibilité d'un objet. Il existe 4 types de portées principales :

- Page : l'objet est visible uniquement dans la page courante
- Request : l'objet est visible pour toute la durée d'une requête HTTP
- Session : l'objet est visible pour toute la durée d'une session utilisateur
- Application : l'objet est visible pour toute l'application web

Problème :

1. Contenu du fichier web.xml :

xml

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
    version="4.0">
    <servlet>
        <servlet-name>CreationDossier</servlet-name>
        <servlet-class>com.example.CreationDossier</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>CreationDossier</servlet-name>
        <url-pattern>/creerDossier</url-pattern>
    </servlet-mapping>
    <servlet>
        <servlet-name>SuppressionDossier</servlet-name>
        <servlet-class>com.example.SuppressionDossier</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>SuppressionDossier</servlet-name>
        <url-pattern>/supprimerDossier</url-pattern>
    </servlet-mapping>
</web-app>

```

2. Création du bean DossierMedical :

java

```

public class DossierMedical {
    private int idPatient;
    private String nom;
    private String prenom;
    private Date dateNaissance;
    private String adresse;
    private String telephone;
    private String historiqueMedical;
    private String traitements;
    private String allergies;
    private String medecin;

    // Getters, setters et constructeurs
}

```

3. Création du formulaire creerDossier.jsp :

html

```

<form action="creerDossier" method="post">
    <label for="nom">Nom :</label>
    <input type="text" id="nom" name="nom" required>

    <!-- Autres champs du formulaire -->

    <button type="submit">Créer le dossier</button>
</form>

```

4. Contrôleur CreationDossier :

java

```

public class CreationDossier extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Récupération des données du formulaire
        String nom = request.getParameter("nom");
        // Récupération des autres paramètres

        // Création d'un nouveau DossierMedical
        DossierMedical dossier = new DossierMedical();
        dossier.setNom(nom);
        // Initialisation des autres champs du dossier

        // Logique de stockage du dossier médical
        // ...

        // Redirection vers la page d'affichage du dossier
        request.setAttribute("dossier", dossier);
        request.getRequestDispatcher("/afficheDossier.jsp").forward(request,
response);
    }
}

```

5. Page afficheDossier.jsp :

html

```
<h1>Dossier médical</h1>
<p>Nom : ${dossier.nom}</p>
<p>Prénom : ${dossier.prenom}</p>
<!-- Affichage des autres champs du dossier médical -->
```

6. Formulaire mettreAJourDossier.jsp :

html

```
<form action="mettreAJourDossier" method="post">
  <label for="nom">Nom :</label>
  <input type="text" id="nom" name="nom" value="${dossier.nom}" required>

  <!-- Autres champs du formulaire pré-remplis avec les valeurs du dossier -->

  <button type="submit">Mettre à jour le dossier</button>
</form>
```

7. Formulaire supprimerDossier.jsp :

html

```
<form action="supprimerDossier" method="post">
  <label for="idPatient">Identifiant du patient :</label>
  <input type="text" id="idPatient" name="idPatient" required>

  <button type="submit">Supprimer le dossier</button>
</form>
```

8. Contrôleur SuppressionDossier :

java

```
public class SuppressionDossier extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Récupération de l'identifiant du patient
        int idPatient = Integer.parseInt(request.getParameter("idPatient"));

        // Logique de suppression du dossier médical
        // ...

        // Redirection vers une page de confirmation ou de liste des dossiers
        response.sendRedirect

```

7. Créer le formulaire supprimerDossier.jsp pour la suppression des dossiers médicaux.

jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"%>
<!DOCTYPE html>
<html>
<head>
```

```
<meta charset="UTF-8">
<title>Supprimer Dossier Médical</title>
</head>
<body>
  <h1>Supprimer Dossier Médical</h1>
  <form action="SuppressionDossierServlet" method="post">
    <label for="patientId">Identifiant du patient :</label>
    <input type="text" id="patientId" name="patientId" required>
    <br>
    <input type="submit" value="Supprimer">
  </form>
</body>
</html>
```

8. Proposer un code pour le contrôleur SuppressionDossierServlet pour gérer la suppression des dossiers médicaux.

java

```
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.servlet.ModelAndView;

@Controller
public class SuppressionDossierServlet {

    @PostMapping("/SuppressionDossierServlet")
    public ModelAndView supprimerDossier(@RequestParam("patientId") String
patientId, HttpServletRequest request, HttpServletResponse response) {
        // Logique de suppression du dossier médical à partir de l'identifiant du
patient

        // Récupération des informations du dossier à supprimer
        String nom = "John Doe";
        String prenom = "John";

        // Création du modèle et de la vue
        ModelAndView modelAndView = new ModelAndView("supprimerDossier");
        modelAndView.addObject("patientId", patientId);
        modelAndView.addObject("nom", nom);
        modelAndView.addObject("prenom", prenom);

        // Redirection vers la page de confirmation de suppression
        return modelAndView;
    }
}
```

Dans cette implémentation, le contrôleur SuppressionDossierServlet est annoté avec @Controller pour indiquer qu'il s'agit d'un contrôleur Spring MVC.

La méthode supprimerDossier() est annotée avec @PostMapping("/SuppressionDossierServlet") pour mapper la requête POST vers cette méthode.

La méthode récupère l'identifiant du patient à partir du paramètre patientId grâce à l'annotation @RequestParam("patientId").

Ensuite, la méthode effectue la logique de suppression du dossier médical à partir de l'identifiant du patient (non implémenté dans cet exemple).

Ou en utilisant les annotations.

1. Contenu du fichier web.xml pour le mapping des fonctionnalités :

```
xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
  version="4.0">
  <servlet>
    <servlet-name>CreationDossier</servlet-name>
    <servlet-class>com.hospital.dme.servlets.CreationDossier</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>CreationDossier</servlet-name>
    <url-pattern>/creerDossier</url-pattern>
  </servlet-mapping>

  <servlet>
    <servlet-name>SuppressionDossier</servlet-name>
    <servlet-class>com.hospital.dme.servlets.SuppressionDossier</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>SuppressionDossier</servlet-name>
    <url-pattern>/supprimerDossier</url-pattern>
  </servlet-mapping>
</web-app>
```

2. Bean DossierMedical :

```
java
public class DossierMedical {
  private int identifiantPatient;
  private String nom;
  private String prenom;
  private Date dateNaissance;
  private String adresse;
  private String numTelephone;
  private String historiqueMedical;
  private String traitements;
  private String allergies;
  private String medecinReferent;

  // Getters, setters, constructeurs
}
```

3. Formulaire creerDossier.jsp :

html

```
<form action="creerDossier" method="post">
  <label for="nom">Nom :</label>
  <input type="text" id="nom" name="nom" required>
  <label for="prenom">Prénom :</label>
  <input type="text" id="prenom" name="prenom" required>
  <label for="dateNaissance">Date de naissance :</label>
  <input type="date" id="dateNaissance" name="dateNaissance" required>
  <label for="adresse">Adresse :</label>
  <input type="text" id="adresse" name="adresse" required>
  <label for="numTelephone">Numéro de téléphone :</label>
  <input type="tel" id="numTelephone" name="numTelephone" required>
  <label for="historiqueMedical">Historique médical :</label>
  <textarea id="historiqueMedical" name="historiqueMedical" required></textarea>
  <label for="traitements">Traitements en cours :</label>
  <input type="text" id="traitements" name="traitements" required>
  <label for="allergies">Allergies :</label>
  <input type="text" id="allergies" name="allergies" required>
  <label for="medecinReferent">Médecin référent :</label>
  <input type="text" id="medecinReferent" name="medecinReferent" required>
  <button type="submit">Créer le dossier</button>
</form>
```

4. Contrôleur CreationDossier (annotation) :

java

```
@WebServlet("/creerDossier")
public class CreationDossier extends HttpServlet {
  protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    // Récupération des données du formulaire
    String nom = request.getParameter("nom");
    String prenom = request.getParameter("prenom");
    Date dateNaissance = Date.valueOf(request.getParameter("dateNaissance"));
    String adresse = request.getParameter("adresse");
    String numTelephone = request.getParameter("numTelephone");
    String historiqueMedical = request.getParameter("historiqueMedical");
    String traitements = request.getParameter("traitements");
    String allergies = request.getParameter("allergies");
    String medecinReferent = request.getParameter("medecinReferent");

    // Création du dossier médical
    DossierMedical dossier = new DossierMedical();
    dossier.setNom(nom);
    dossier.setPrenom(prenom);
    dossier.setDateNaissance(dateNaissance);
    dossier.setAdresse(adresse);
    dossier.setNumTelephone(numTelephone);
    dossier.setHistoriqueMedical(historiqueMedical);
    dossier.setTraitements(traitements);
    dossier.setAllergies(allergies);
    dossier.setMedecinReferent(medecinReferent);

    // Redirection vers la page d'affichage du dossier
    request.setAttribute("dossier", dossier);
    request.getRequestDispatcher("/afficheDossier.jsp").forward(request,
    response);
  }
}
```


EXAMEN SESSION RATTRAPAGE SEMESTRE 2

Année Académique : 2023/2024 – INF 332- Ingénierie des applications web – IN3 Durée : 3H

5. Page afficheDossier.jsp :

```
<h1>Dossier médical de ${dossier.prenom} ${dossier.nom}</h1>
<p>Date de naissance : ${dossier.dateNaissance}</p>
<p>Adresse : ${dossier.adresse}</p>
<p>Numéro de téléphone : ${dossier.numTelephone}</p>
<p>Historique médical : ${dossier.historiqueMedical}</p>
<p>Traitements en cours : ${dossier.traitements}</p>
<p>Allergies : ${dossier.allergies}</p>
<p>Médecin référent : ${dossier.medecinReferent}</p>
```

6. Formulaire mettreAJourDossier.jsp :

```
<form action="mettreAJourDossier" method="post">
  <input type="hidden" name="identifiantPatient" value="${dossier.identifiantPatient}">
  <label for="nom">Nom :</label>
  <input type="text" id="nom" name="nom" value="${dossier.nom}" required>
  <label for="prenom">Prénom :</label>
  <input type="text" id="prenom" name="prenom" value="${dossier.prenom}"
  required>
  <label for="dateNaissance">Date de naissance :</label>
  <input type="date" id="dateNaissance" name="dateNaissance" value="${dossier.dateNaissance}" required>
  <label for="adresse">Adresse :</label>
  <input type="text" id="adresse" name="adresse" value="${dossier.adresse}"
  required>
  <label for="numTelephone">Numéro de téléphone :</label>
  <input type="tel" id="numTelephone" name="numTelephone" value="${dossier.numTelephone}" required>
  <label for="historiqueMedical">Historique médical :</label>
  <textarea id="historiqueMedical" name="historiqueMedical" required>${dossier.historiqueMedical}</textarea>
  <label for="traitements">Traitements en cours :</label>
  <input type="text" id="traitements" name="traitements" value="${dossier.traitements}" required>
  <label for="allergies">Allergies :</label>
  <input type="text" id="allergies" value="${dossier.allergies}">
```

Exercice 1 : (4,5 points)

1. Définir les termes suivants : RESTful API, Microservices (1 pt)
2. Qu'est-ce qu'un middleware dans le contexte des applications web et pourquoi est-il important ? (1 pt)
3. Expliquer comment gérer l'authentification et l'autorisation dans une application web. Fournir un exemple de code pour une authentification basique. (1,5 pts)
4. Décrire la structure d'une requête HTTP et expliquer les différentes parties. (1 pt)

Exercice 2 : (4,5 points)

1. Décrire le rôle des tokens dans la gestion des sessions et comment ils diffèrent des cookies. (1,5 pts)
2. Écrire un exemple de code pour créer, lire et supprimer un token d'authentification dans une application web. (1,5 pts)
3. Définir le terme "scope" dans le contexte des applications web et citer 4 types de scopes utilisés couramment. (1,5 pts)

Problème : (11 points) Contexte : Vous travaillez sur un projet de gestion des employés pour une entreprise. Vous devez créer un module applicatif pour la création, la mise à jour, la suppression et la visualisation des informations des employés. Les fonctionnalités attendues sont les suivantes :

- Création d'un employé (créerEmploye.jsp) : Permet de créer un employé en saisissant des données depuis un formulaire.
- Mise à jour d'un employé (mettreAJourEmploye.jsp) : Permet de mettre à jour un employé existant.
- Suppression d'un employé (supprimerEmploye.jsp) : Permet de supprimer un employé en saisissant l'identifiant.
- Affichage des informations d'un employé (afficheEmploye.jsp) : Permet de visualiser les informations de l'employé créé.

Attributs d'un employé : identifiant, nom, prénom, date de naissance, adresse, numéro de téléphone, département, poste, date d'embauche.

1. Proposer le contenu du fichier web.xml pour effectuer le mapping de nos différentes fonctionnalités. (1,5 pts)
2. Créer le bean Employe. (2 pts)
3. Créer le formulaire créerEmploye.jsp qui redirige vers afficheEmploye.jsp. (2 pts)
4. Proposer un code pour le contrôleur CreationEmploye pour gérer les fonctionnalités des employés. (2 pts)
5. Créer la page afficheEmploye.jsp. (2 pts)
6. Créer le formulaire mettreAJourEmploye.jsp pour la mise à jour des informations des employés. (2 pts)
7. Créer le formulaire supprimerEmploye.jsp pour la suppression des employés. (2 pts)
8. Proposer un code pour le contrôleur SuppressionEmploye pour gérer la suppression des employés. (2 pts)

EXAMEN NORMALE SEMESTRE 2

Année Académique : 2022/2023 - INF 332- Ingénierie des applications web - IN3 Durée : 3H

Exercice 1 (4,5 pts)

- 1) Définir les termes suivants : DAO, servlet. (1 pt)
- 2) Quelles sont les principales annotations utilisées dans une servlet ? Identifier deux et préciser le rôle de chacune et sa syntaxe. (1,5 pts)
- 3) Donner 2 avantages et 2 inconvénients d'une architecture multicouches ? (1 pt)
- 4) Donner la Structure d'une servlet HTTP. (1 pt)

Exercice 2 (4,5 pts)

- 1) C'est quoi un Java Bean ? Pourquoi les beans sont utilisés dans l'architecture JAVA EE au lieu d'écrire tout le code dans les JSP ? (1,5 pts)
- 2) Quelles sont les directives de JSP ? Pour chacune préciser son rôle et sa syntaxe d'utilisation. (1,5 pts)
- 3) Citez les types de portes dans une application. Citez 4 types de portes. (1,5 pts)

Problème (11 pts)

Dans le cadre des activités d'un système de gestion des offres d'emploi, les principales activités peuvent être divisées comme suit : la gestion des utilisateurs, gestion des tâches. La gestion des utilisateurs est une activité essentielle au fonctionnement du système, car elle permet aux candidats de créer et de gérer des utilisateurs, d'accéder à leur profil d'utilisateur, de rechercher des emplois, de postuler à des emplois et de suivre l'état de leurs candidatures. Les administrateurs sont chargés de gérer la base de données des utilisateurs et d'assurer la sécurité du système. La gestion des offres d'emploi est une activité essentielle dans le fonctionnement d'une plateforme de recrutement, qui permet aux employeurs de publier des offres d'emploi, de gérer les candidatures reçues et de sélectionner les candidats pour les entretiens, ainsi qu'aux candidats de rechercher des offres d'emploi et de postuler à celles qui les intéressent. Le module de publication des offres d'emploi permet aux employeurs de créer et de publier des offres d'emploi, d'ajouter des informations détaillées sur le poste, de préciser la date limite de dépôt des candidatures et le processus de sélection des candidats, de gérer les candidatures reçues et de sélectionner les candidats pour les entretiens, et de communiquer avec les candidats pour organiser des entretiens ou leur donner des informations en retour sur leur candidature. Le module de gestion des candidatures permet aux employeurs de trier les candidatures en fonction de critères tels que l'expérience professionnelle, les compétences et la formation, et de communiquer avec les candidats pour organiser des entretiens et donner un retour d'information sur leur candidature.

Pour la création du Utilisateur et postuler à un emploi, vous devez utiliser respectivement les fichiers `creerUtilisateur.jsp`, `creerEmploi.jsp` et `postuler.jsp`. Un User est décrit par son id, nom, email et pays. Un Employeur est décrit par son id, nomEntreprise, status social, email, phone et password.

- **Création d'un utilisateur :** elle permet de créer un utilisateur en saisissant des données depuis un formulaire, et visualiser la fiche utilisateur en résultant. Utiliser un bean, représentant un utilisateur ; une servlet appelée `CreationUtilisateur` chargée de récupérer les données envoyées par le formulaire, de les enregistrer dans le bean et de les transmettre à une JSP ;
- **Afficher le Utilisateur :** chargée d'afficher la fiche du utilisateur créé, c'est-à-dire les données transmises par la servlet.
- **Création d'une offre d'emploi :** elle permet de créer un emploi en saisissant des données depuis un formulaire, et visualiser le résultat. Utiliser un bean, représentant une offre, `offreEmploi` ; une servlet appelée `CreerOffre`, chargée de récupérer les données envoyées par le formulaire, de les enregistrer dans le bean et de les transmettre à une JSP ;

- **Postuler :** l'utilisateur doit également pouvoir postuler à un emploi, en remplissant le formulaire de la figure 1, la servlet appelée `Application`, chargée de récupérer les données envoyées par le formulaire, et de les transmettre à notre vue `afficheApplication.jsp` ;
- 1 Proposer le contenu du fichier `web.xml` pour effectuer le mapping pour nos différentes fonctionnalités. (1,5 pts)
- 2 Créer les deux beans `utilisateur` et `offreEmploi`. (1,5 pts)
- 3 Réaliser la fonctionnalité créer nouveau emploi. (2,5 pts)
- 3.1 Donner le code de `creerEmploi.jsp`. (1 pt)
- 3.2 Produire le code du contrôleur `CreerOffre` qui affiche en sa méthode `doGet` `creerEmploi.jsp` et en `doPost` enregistre l'emploi dans un bean et redirige vers `afficheEmploi.jsp`. (1,5 pt)
- 4 Proposer un code pour le contrôleur `Application` pour gérer les fonctionnalités des Candidatures des utilisateurs aux offres d'emplois. (2 pts)
- 5 Créer un code pour la vue `postuler.jsp` en se servant de la figure 1. Donner son code source et mettre à jour les annotations ou le fichier `web.xml`. (2 pts)
- 6 Pour étendre les fonctionnalités de notre mini application, proposez l'intégration de la couche d'accès aux données que vous utiliserez dans le contrôleur `Application` pour enregistrer une candidature faite au niveau de la vue `postuler.jsp` dans une base de données Mysql une fois créé. (2 pts)

Candidature pour une offre d'emploi

Veuillez remplir le formulaire ci-dessous pour postuler à une offre d'emploi.

Nom complet :

Prénom : Nom de famille :

Adresse actuelle :

Adresse E-mail :

Numéro de téléphone :

Candidature au poste no. :

Type de contrat :

Deposez votre CV / et lettre de motivation ici :

Parcourir les fichiers :

Figure 1: Formulaire pour postuler à une offre d'emploi

Exercice 1 (4 pts)

1. Citer deux avantages et deux inconvénients d'une architecture multicouche? (1 pt)
2. Dans le domaine des serveurs d'applications, citer une solution propriétaire et payante, et une solution libre et gratuite. (1 pt)
3. Donner l'arborescence des fichiers d'une application web de type JEE. (1 pt)
4. Donner la signature de la méthode d'une servlet qui traite une commande GET du protocole HTTP? (1 pt)

Exercice 2 (5 pts)

1. Quels sont les éléments constitutifs d'un serveur d'application? (1pt)
2. Quelle est la différence entre l'architecture client-serveur et le modèle MVC? (1pt)
3. Décrire par un schéma le modèle MVC sous le prisme de Java EE. Le schéma devra contenir les différentes couches de Java EE et préciser leur rôle. (1,5pts)
4. Définir le terme portée dans une application. Citer quatre types de portées. (1,5pts)

Problème (11 pts)

Dans ce problème, l'utilisation des tags JSTL est préférable pour écrire les JSPs. Pour le mapping des tables de la base de données, vous pouvez utiliser JDBC. L'objectif est de développer une application Web permettant de gérer des contacts (créer un contact, rechercher un contact, modifier un contact, ...). Cette application doit être conforme au design pattern MVC (i.e. des JSPs pour les vues, une Servlet comme contrôleur et des classes Java (JDBC) pour le modèle).

Pour cela, on crée des JSPs pour afficher un contact, éditer un contact, ... Chaque une de ces JSPs communiquera avec une Servlet qui aura le rôle de contrôleur : *ControllerServlet.java*. L'idée ici est que toutes vos requêtes doivent être sous la forme: *ControllerServlet?do_this=[action à exécuter]*, le paramètre *do_this* est récupéré par la *ControllerServlet* pour connaître l'action à effectuer (par exemple, sélectionner la JSP qui aura à afficher la réponse).

- Partie Modèle:

Vous devez commencer par créer la table Contact:

CREATE TABLE Contact (

ID_CONTACT generated always as identity primary key,
 FIRSTNAME VARCHAR(255),
 LASTNAME VARCHAR(255),
 EMAIL VARCHAR(255),
 PHONE INTEGER,
 ADDRESS VARCHAR(255),
 PROFESSION VARCHAR(125)

page
 servet
 application

Dans le projet on suppose avoir un package nommé *model* sous le répertoire *src*. Il est nécessaire d'implémenter la classe *Contact* (un simple *JavaBean* à mapper avec *JDBC*) correspondant au mapping de la table *Contact* et la classe *ContactFacade.java* dans le package *model*.

- Partie Contrôleur:

Dans un package nommé *controller*, une servlet *ControllerServlet.java* implémente les fonctions provenant des besoins des JSPs développées dans la partie Vues.

- Partie Vues:

L'utilisateur arrive sur une page d'accueil *accueil.jsp*. Cette page affiche la liste des contacts déjà créés. En plus, la page *accueil.jsp* contient les actions suivantes (liens):

- Créer un nouveau contact (redirection vers *addContact.jsp*)
- Supprimer un contact (redirection vers *removeContact.jsp*)
- Modifier un contact (redirection vers *updateContact.jsp*)
- Rechercher un contact (redirection vers *searchContact.jsp*)

La page *accueil.jsp* se trouve dans le répertoire */Webapp* de votre application en supposant vous utilisez Eclipse. Cette page utilise le tag *JSTL* dans le but de générer dynamiquement un tableau HTML sans utiliser une boucle «for» en code Java.

Attention: les JSPs ne peuvent pas accéder directement à la base de données car dans MVC les Vues n'appellent pas directement le Modèle. En effet, tous les appels vers le Modèle (à travers *ContactFacade.java*) doivent passer par le Contrôleur *ControllerServlet.java*. Par exemple, pour accéder à la page d'accueil *accueil.jsp*, l'utilisateur doit appeler la *ControllerServlet* (URL: *http://localhost:8080/Contacts/ControllerServlet*). Ensuite, la *ControllerServlet* transmet la requête vers *accueil.jsp*. Enfin, *accueil.jsp* récupère la liste des contacts à partir de la requête et l'affiche.

La page *addContact.jsp* propose un formulaire qui contient des champs pour renseigner un contact. Ses champs doivent correspondre aux attributs de la classe *Contact.java*. Le formulaire doit également avoir un bouton «submit» et un bouton «annuler». Une fois le formulaire envoyé, il est traité par la servlet *ControllerServlet*. Par conséquent, il faut préciser à *ControllerServlet* l'action à effectuer.

La page *addContact.jsp* doit être créée dans *Contacts/webapp/*. Ensuite, la Servlet *ControllerServlet* récupère les informations du contact à partir des données du formulaire puis elle utilise le *ContactFacade* pour appeler la méthode *addContact(long id, String firstname, String lastname, String email, ...)*. Cette méthode crée une instance de *Contact*, fixe ses attributs puis le sauvegarde dans la base de données. Même principe pour *removeContact.jsp*. Cette page propose un formulaire avec un champ nommé «idContact» et un bouton «submit». Une fois cliqué, l'utilisateur est invité à confirmer la suppression par une boîte de dialogue javascript une fois confirmée, le formulaire fait appel à la servlet *ControllerServlet* qui récupère l'id du contact à partir des données du formulaire puis elle appelle la méthode *deleteContact(long id)* de *ContactFacade*. Cette méthode se charge d'effacer le contact portant cet id dans la base de données. Ainsi de suite pour les autres fonctionnalités.

1. Proposer un contenu du fichier *web.xml* pour effectuer le mapping pour des différentes fonctionnalités : « afficher contacts », « nouveau contact », « recherche » et « modifier contact ». (1,5 pts)
2. Créer le bean représentant un contact et la classe *ContactFacade* pour l'accès à la base de données (avec ses différentes méthodes). (2,5 pts)
3. Créer le formulaire un « nouveau contact » (redirection vers *addContact.jsp*). (1,5 pts)
4. Créer la page modifier contact (redirection vers *updateContact.jsp*). (1,5 pts)
5. Créer la page « Home contact » (redirection vers *accueil.jsp*) (2 pts)
6. Proposer un code pour le contrôleur *ControllerServlet* pour gérer les fonctionnalités du projet (2 pts)

Exercice 1 (4 pts)

1. Citer deux avantages et deux inconvénients d'une architecture multicouche? (1 pt)

Deux principaux avantages que procure l'élaboration d'une architecture « multi-couches » sont:

- Structure de l'application adaptée à l'architecture de déploiement ciblée,
- Modularité de l'application, -
- Factorisation du code avec possibilité d'exploiter un framework ou des composants génériques

(gain de temps et de performances).

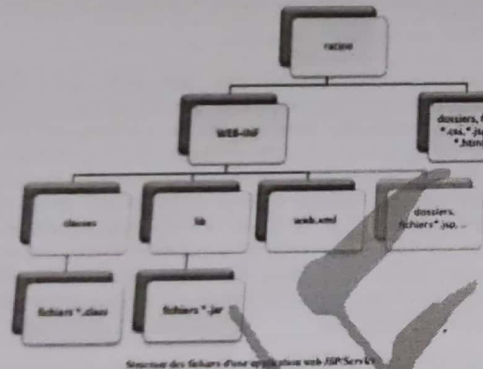
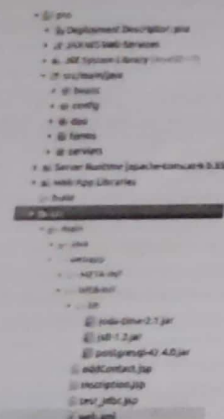
Les inconvénients majeurs sont :

- L'augmentation de la complexité lors de rajout de plusieurs services,
- Nécessité des connaissances tant technique que théorique.

2. Dans le domaine des serveurs d'applications, citer une solution propriétaire et payante, et une solution libre et gratuite. (1 pt)

- Tomcat, Wilfly
- WebSphere

3. Donner l'arborescence des fichiers d'une application web de type JEE. (1 pt)



4. Donner la signature de la méthode d'une servlet qui traite une commande GET du protocole HTTP ? (1 pt)

```
public void doGet(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
    // add code du traitement
    this.getServletContext().getRequestDispatcher(VUE).forward(request,
response);
}
```

Exercice 2 (5 pts)

1. Quels sont les éléments constitutifs d'un serveur d'application ? (1pt)

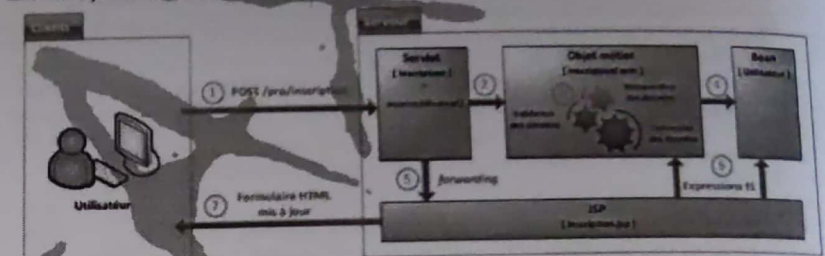
- Serveur HTTP
- Conteneur web

2. Quelle est la différence entre l'architecture client-serveur et le modèle MVC ? (1pt)

L'architecture client-serveur est une forme de codage système qui rend un serveur connecté aux clients, tandis que MVC est un modèle de conception qui peut être implémenté sur une architecture client-serveur pour en faciliter l'organisation.

3. Décrire par un schéma le modèle MVC sous le prisme de Java EE. Le schéma devra contenir les différentes couches de Java EE et préciser leur rôle. (1,5pts)

Voici sous forme d'un schéma ce que nous avons réalisé :



4. Définir le terme portée dans une application. Citer quatre types de portées (1,5pts)

Il existe 04 portées :

- La page (JSP seulement) : les objets possédants cette portée sont uniquement accessible dans la page JSP en question.
- La requête : les objets possédants cette portée sont uniquement accessible durant l'existence de la requête en cours.
- La session : les objets possédants cette portée sont uniquement accessible durant l'existence de la session en cours.
- L'application : les objets possédants cette portée sont accessibles durant l'existence de l'application.

Problème

CREATE TABLE public."Contact"

```
(
    id integer NOT NULL, firstname character varying(255) NOT NULL, lastname character
varying(255) NOT NULL, email character varying(100) NOT NULL, phone integer NOT
NULL, address character varying(255), profession character varying(125),
CONSTRAINT id_contact PRIMARY KEY (id),
CONSTRAINT mail UNIQUE (email),
CONSTRAINT tel UNIQUE (phone)
)
```

```
WITH (
    OIDS = FALSE
);
```

```
ALTER TABLE public."Contact"
OWNER to postgres;
```

```
package beans;
public class Contact {
```

```
    Integer id,phone;
    String firstname, lastname, email, address, profession;
    public Integer getId() {
```



```

        return id;
    }
    public void setId(Integer id) {
        this.id = id;
    }
    public String getFirstname() {
        return firstname;
    }
    public void setFirstname(String firstname) {
        this.firstname = firstname;
    }
    public String getLastName() {
        return lastname;
    }
    public void setLastName(String lastname) {
        this.lastname = lastname;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public Integer getPhone() {
        return phone;
    }
    public void setPhone(Integer phone) {
        this.phone = phone;
    }
    public String getAddress() {
        return address;
    }
    public void setAddress(String address) {
        this.address = address;
    }
    public String getProfession() {
        return profession;
    }
    public void setProfession(String profession) {
        this.profession = profession;
    }
}

package beans;
import java.util.List;
import dao.DAOException;
public interface ContactFacade {
    void creer( Contact contact ) throws DAOException;
    Contact trouver( String email ) throws DAOException;
    List<Contact> lister() throws DAOException;
    void supprimer( Contact contact ) throws DAOException;
}

package beans;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

import dao.DAOException;

```

```

import dao.DAOFactory;
import dao.DAOUtilitaire;

public class ContactFacadeImpl implements ContactFacade {
    public static final String TABLE = "public.\"Contact\"";
    private static final String SQL_SELECT = "SELECT id, firstname, lastname, email, phone, address, profession FROM " + TABLE + " ORDER BY id";
    private static final String SQL_SELECT_PAR_ID = "SELECT id, firstname, lastname, email, phone, address, profession FROM " + TABLE + " WHERE id = ?";
    private static final String SQL_INSERT = "INSERT INTO " + TABLE + " (id, firstname, lastname, email, phone, address, profession) VALUES (?, ?, ?, ?, ?, ?, ?)";
    private static final String SQL_DELETE_PAR_ID = "DELETE FROM FROM " + TABLE + " WHERE id = ?";
    private DAOFactory daoFactory;

    public ContactFacadeImpl(DAOFactory daoFactory) {
        this.daoFactory = daoFactory;
    }

    @Override
    public void creer(Contact contact) throws DAOException {
        Connection connexion = null;
        PreparedStatement preparedStatement = null;
        ResultSet valeursAutoGenerees = null;
        try {
            connexion = daoFactory.getConnection();
            preparedStatement = DAOUtilitaire.initialisationRequetePrepree(connexion, SQL_INSERT, true, contact.getId(), contact.getFirstname(), contact.getLastName(), contact.getEmail(), contact.getPhone(), contact.getAddress(), contact.getProfession());
            int statut = preparedStatement.executeUpdate();
            if (statut == 0) {
                throw new DAOException("Échec de la création du contact, aucune ligne ajoutée dans la table.");
            }
            valeursAutoGenerees = preparedStatement.getGeneratedKeys();
            if (valeursAutoGenerees.next()) {
                contact.setId(valeursAutoGenerees.getInt(1));
            } else {
                throw new DAOException("Échec de la création du contact en base, aucun ID auto-généré retourné.");
            }
        } catch (SQLException e) {
            throw new DAOException(e);
        } finally {
            DAOUtilitaire.fermeturesSilencieuses(valeursAutoGenerees, preparedStatement, connexion);
        }
    }

    @Override
    public Contact trouver(String email) throws DAOException {
        // TODO Auto-generated method stub
        return null;
    }

    @Override

```

```

public List<Contact> lister() throws DAOException {
    Connection connexion = null;
    PreparedStatement preparedStatement = null;
    ResultSet resultSet = null;
    List<Contact> contacts = new ArrayList<>();
    try {
        connexion = daoFactory.getConnection();
        preparedStatement = connexion.prepareStatement(SQL_SELECT);
        resultSet = preparedStatement.executeQuery();
        while (resultSet.next()) {
            contacts.add(map(resultSet));
        }
    } catch (SQLException e) {
        throw new DAOException(e);
    } finally {
        DAOUtilitaire.fermerResiliences(resultSet,
            preparedStatement, connexion);
    }
    return contacts;
}

@Override
public void supprimer(Contact contact) throws DAOException {
    Connection connexion = null;
    PreparedStatement preparedStatement = null;
    try {
        connexion = daoFactory.getConnection();
        preparedStatement = connexion.prepareStatement(SQL_DELETE_PAR_ID, true,
            DAOUtilitaire.initialiserRequetePrepree(connexion, SQL_DELETE_PAR_ID, true,
                "id statut = 0"));
        preparedStatement.executeUpdate();
        if (statut == 0) {
            throw new DAOException("Echec de la suppression du
                contact, aucune ligne supprimée de la table.");
        } else {
            contact.setId(null);
        }
    } catch (SQLException e) {
        throw new DAOException(e);
    } finally {
        DAOUtilitaire.fermerResiliences(preparedStatement,
            connexion);
    }
}

/*
 * Simple méthode utilitaire permettant de faire la correspondance (le
 * mapping)
 * ENTRE une ligne issue de la table des contacts (un ResultSet) et un
 * bean
 */
Contact;

private static Contact map(ResultSet resultSet) throws SQLException {
    Contact contact = new Contact();
    contact.setId(resultSet.getInt("id"));
    contact.setFirstName(resultSet.getString("firstName"));
    contact.setLastName(resultSet.getString("lastName"));
    contact.setAddress(resultSet.getString("address"));
    contact.setPhone(resultSet.getString("phone"));
    contact.setEmail(resultSet.getString("email"));
    contact.setProfession(resultSet.getString("profession"));
    return contact;
}

```

```

}

package servlets;

import java.io.IOException;
import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import beans.Contact;
import beans.ContactFacade;
import dao.DAOFactory;

/**
 * Servlet implementation class ControllerServlet
 */
@WebServlet("/ControllerServlet")
public class ControllerServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    private static final String CONF_DAO_FACTORY = "daoFactory";
    ContactFacade contactFacade;

    /**
     * @see HttpServlet#HttpServlet()
     */
    public ControllerServlet() {
        super();
        // TODO Auto-generated constructor stub
    }

    public void init(ServletConfig config) throws ServletException {
        System.out.println("*** initialising controller servlet.");
        super.init(config);
        /* Récupération d'une instance de notre DAO */
        this.contactFacade = (DAOFactory)
            getServletContext().getAttribute(CONF_DAO_FACTORY).getContactDao();
    }

    /**
     * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse
     * response)
     */
    protected void doGet(HttpServletRequest request, HttpServletResponse
        response)
        throws ServletException, IOException {
        doPost(request, response);
    }

    /**
     * @see HttpServlet#doPost(HttpServletRequest request, HttpServletResponse
     * response)
     */
    protected void doPost(HttpServletRequest request, HttpServletResponse
        response) throws ServletException, IOException {

```



```

// recuperation de l'action à effectuer
String do_this = request.getParameter("do_this");
if (do_this == null) {
    // definir le contexte pour une redirection sur la page accueil.jsp
    ServletContext sc = getServletContext();
    RequestDispatcher rd = sc.getRequestDispatcher("/accueil.jsp");
    // charger la liste des contacts dans la requête pour les
    // transmettre à la jsp accueil.jsp (qui va les afficher)
    request.setAttribute("listContacts", contactFacade.lister());
    rd.forward(request, response);
} else if (do_this.equals("delete")) {
    // recuperation de l'id du contact
    String id = (String) request.getParameter("contact_id");
    if (id == null) {
        // redirection sur la page removeContact.jsp
        response.sendRedirect("removeContact.jsp");
    } else {
        // id non nul, donc on supprime le contact identifié par id
        contactFacade.supprimer(Integer.parseInt(id));
        // on recharge la page d'accueil
        response.sendRedirect("ControllerServlet");
    }
} else if (do_this.equals("create")) {
    // recuperation du nom contact
    String lastName = (String) request.getParameter("lastName");
    if (lastName == null) {
        // redirection sur la page removeContact.jsp
        response.sendRedirect("addContact.jsp");
    } else {
        // TODO le nom n'est pas nul, donc on ajoute le contact dans la base
        Contact contact = new Contact();
        contactFacade.creer(contact);
        // on recharge la page d'accueil
        response.sendRedirect("ControllerServlet");
    }
} else if (do_this.equals("update")) {
    // TODO ...
} else if (do_this.equals("search")) {
    // TODO ...
}
// TODO ...
}

}

page d'accueil accueil.jsp:
<%page import="beans.Contact"%>
<%page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Accueil</title>
</head>
<body>
<h1>Liste des contacts :</h1>
<table border="2">
<tr>
<th>Contact Id</th>

```

```

<th>First Name</th>
<th>Last Name</th>
<th>Email</th>
<th>Phone Number</th>
<th>Address</th>
</tr>
<:forEach var="contact" items="${requestScope.listContacts}">
<tr>
<td>...</td>
<td>...</td>
<td>...</td>
<td>...</td>
<td>...</td>
</tr>
</:forEach>
</table>
<a href="ControllerServlet?do_this=create" style="">Cr  er un
    nouveau contact</a>
<a href="ControllerServlet?do_this=delete" style="">Supprimer un
    Contact</a>
<a href="ControllerServlet?do_this=update" style="">Modifier un
    Contact </a>
<a href="ControllerServlet?do_this=search" style="">Rechercher un
    contact </a>
</body>
</html>

Page addContact.jsp
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Add contact</title>
</head>
<body>
<form action="ControllerServlet" method="POST">
<input type="hidden" name="do_this" value="create" />
<table>
<tr>
<td align="center" colspan="2"><font size="4">Please Enter the following
    Details</font>
</tr>
<tr>
<td><input type="text" name="firstName"/></td></tr>
<tr>
<td><input type="text" name="lastName"/></td></tr>
</table>
</tr>
<tr>
<td align="center" colspan="2"><input type="submit" name="validate" value="save"/></td>
</tr>
</table>
</body>
</html>

Page updateContact.jsp
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Add contact</title>
</head>

```

```

<body>
<form action="ControllerServlet" method="POST">
<input type="hidden" name="do_this" value="update" />
<table>
<tr>
<td align="center" colspan="2"><font size="4">Please Enter the Following
Details</font>
</td></tr>
<tr><td>First Name</td><td><input type="text" name="firstName"/></td></tr>
<tr><td>Last Name</td><td><input type="text" name="lastName"/><br/></td></tr>
<tr>
<td></td><td><input type="submit" name="Validate" value="save"/></td></tr>
</table>
</form>
</body>
</html>

```


Exercice 1 (6 pts)

- 1) Dans quel répertoire se situe le fichier web.xml d'un Dynamic Web Project ? quel est le rôle de ce fichier ? (1 pt)
- 2) Donnez deux différences fondamentales entre l'envoi des requêtes par la méthode Post et l'envoi des requêtes par la méthode Get. (1 pt)
- 3) Toute classe java pour être considérée comme servlet doit impérativement hériter d'une autre classe. Laquelle ? (donner le nom de cette classe en précisant le package dans lequel elle se trouve). (1 pt)
- 4) Combien de paramètres prend la méthode doGet() ? précisez-les en insistant sur leurs types. (1 pt)
- 5) Citez, exemples à l'appui deux directives JSP à votre connaissance. (1 pt)
- 6) Dire en quoi consiste la portée session dans une application. (1 pt)

Exercice 2 (4 pts)

- 1) Décrire le modèle MVC avec java EE par un schéma et dire en quoi consiste chacune des couches qui le constitue. (2 pts)
- 2) Quelle différence faites-vous entre l'architecture client-serveur et le modèle MVC ? (2 pts)

Problème (10 pts)

L'objectif de ce problème est de vérifier votre aptitude à pouvoir créer une application web en utilisant la technologie Java EE. La thématique abordée ici est celle du commerce en ligne. Vous allez créer un embryon d'application qui va permettre la création et la visualisation de commandes. A travers notre application, l'utilisateur doit pouvoir créer une commande en saisissant des données depuis un formulaire et visualiser la fiche commande en résultant.

La création d'une commande se fait en remplissant le formulaire suivant :

Informations commande	
Date *	
Montant *	499.90
Mode de paiement *	Chèque
Statut du paiement	
Mode de livraison *	48H chrono
Statut de la livraison	



Valider

En cas de succès, la page suivante s'affiche à l'utilisateur :

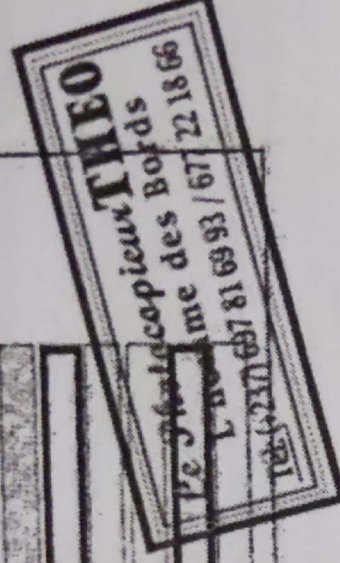
Date : 14/06/2012 10:37:16
Montant : 499.9
Mode de paiement : Chèque
Statut du paiement :
Mode de livraison : 48H chrono
Statut de la livraison :

Et en cas d'échec (Saisie d'un montant erroné par exemple) :

Informations commande

Date " "
Montant "
Mode de paiement "
Statut du paiement "
Mode de livraison "
Statut de la livraison

abdef
Chèque



valider. Remettre à zéro.

On suppose que le projet Web créé à cet effet est nommé TPIN3, que le formulaire de saisi des informations du client est dans le fichier creerCommande.jsp et qu'il peut être accessible à tout le monde. La page qui affiche la confirmation de la création d'un client ne doit pas pouvoir être accessible directement à partir d'une URL du navigateur, elle doit se nommer confirmCommande.jsp. L'envoi des requêtes du formulaire se fait par la méthode Post.

- 1) Sachant que la servlet Commande.java devra être accessible via l'url /commande, donner le contenu qu'il faudra ajouter au fichier web.xml pour effectuer le mapping entre cette url et la servlet. La servlet devra se trouver dans le package com.tp.in3. (2 pts)
- 2) Créer le formulaire creerCommande.jsp. (2 pts)
- 3) Créer le bean représentant les données récupérées depuis le formulaire creerCommande.jsp. (2 pts)
- 4) Créer la servlet Commande.java chargée de récupérer les données envoyées par le formulaire, de les enregistrer dans le bean et de les transférer à confirmCommande.jsp. (2 pts)
- 5) Créer la JSP confirmCommande.jsp chargée de récupérer les données envoyées depuis la servlet Commande.java. (2 pts)

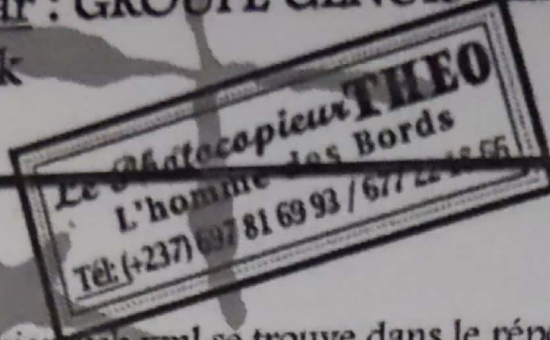
CORRECTION EXAMEN DE FIN DE SECOND SEMESTRE

(NORMALE 2020-2021)

INGENIERIE DES APPLICATION WEB

Proposez par : GROUPE GENUIS REPETITION

Par : Joël_yk



Exercice 1 :

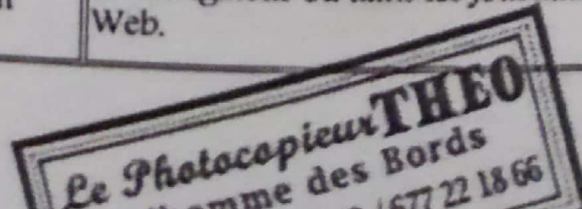
1) Le fichier web.xml se trouve dans le répertoire WEB-INF.

Rôle : le fichier web.xml est le fichier de description ou de configuration de la servlet, il permet de déployer l'application sur une plateforme d'exécution.

2) Différence entre les commandes GET et POST : la différence fondamentale entre les méthodes GET et POST est que selon un critère de bonne pratique la méthode POST doit être utilisée pour réaliser les opérations qui ont un effet sur la ressource, alors que la méthode GET est la méthode utilisée par le client pour récupérer une ressource web au serveur via une url.

➤ Un tableau plus explicatifs :

GET	POST
Dans le cas d'une requête GET, seule une quantité limitée de données peut être envoyée car les données sont envoyées dans l'en-tête.	En cas de requête POST, une grande quantité de données peut être envoyée car les données sont envoyées dans le corps.
La requête GET n'est pas sécurisée car les données sont exposées dans la barre d'URL.	La requête POST est sécurisée car les données ne sont pas exposées dans la barre d'URL.
La requête GET est plus efficace et utilisée plus que POST.	La requête POST est moins efficace et utilisée moins que GET.
Les paramètres restent dans l'historique du navigateur car ils font partie de l'URL.	Les paramètres ne sont pas enregistrés dans l'historique du navigateur.
Les requêtes GET sont ré-exécutées mais ne peuvent pas être soumises au serveur si le code HTML est stocké dans la mémoire cache du navigateur.	Le navigateur prévient généralement l'utilisateur que les données devront être soumises à nouveau.
Seuls les caractères ASCII sont autorisés.	Pas de restrictions. Les données binaires sont également autorisées.
GET est moins sécurisé que POST car les données envoyées font partie de l'URL. Donc, il est enregistré dans l'historique du navigateur et les journaux du serveur en texte brut.	POST est un peu plus sûr que GET car les paramètres ne sont pas stockés dans l'historique du navigateur ou dans les journaux du serveur Web.



GET	POST
La méthode GET ne doit pas être utilisée lors de l'envoi de mots de passe ou d'autres informations sensibles.	Méthode POST utilisée lors de l'envoi de mots de passe ou d'autres informations sensibles.
La méthode GET est visible par tout le monde (elle sera affichée dans la barre d'adresse du navigateur) et limite le nombre d'informations à envoyer.	Les variables de méthode POST ne sont pas visibles dans l'URL.
Peut être mis en cache	Ne peut être mis en cache

- 3) Une Servlet est un composant web conçu sous la forme d'une classe Java. Elle fonctionne dans un moteur/conteneur de servlet dans un serveur d'application. L'objectif d'une servlet est de rendre les contenus web dynamiques. L'API servlet contient des classes et interfaces permettant le développement de servlet.
Packages servlet : javax.servlet

- 4) Cette méthode est invoquée quand le client :

- Saisit directement l'URL dans la barre d'adresse

`http://monserveur/servlet/MaServlet`

- Clique sur un lien hypertext correspondant à l'URL précédente

La méthode `doGet()` possède 02 paramètres : 01 de type `HttpServletRequest` et l'autre de type `HttpServletResponse`.

5) 02 Directives JSP :

La Directive taglib : JSP définit un ensemble de balises (action tags) pour réaliser des actions simples. Exemple :

```
<%@ taglib uri="maTagLib.tld" prefix="tagExemple" %>
```

La Directive page : La directive page définit des informations relatives à la page JSP. Voici par exemple comment importer des classes Java :

Exemple :

```
<%@ page import="java.util.List, java.util.Date" %>
```

La Directive include : la technologie JSP met à notre disposition une balise qui inclut le contenu d'un autre fichier dans le fichier courant. Ceci permet d'inclure une page interne à notre application (en l'occurrence une page JSP nommée `myJSP`, mais cela pourrait très bien être une page HTML ou autre) dans notre JSP courante.

Exemple

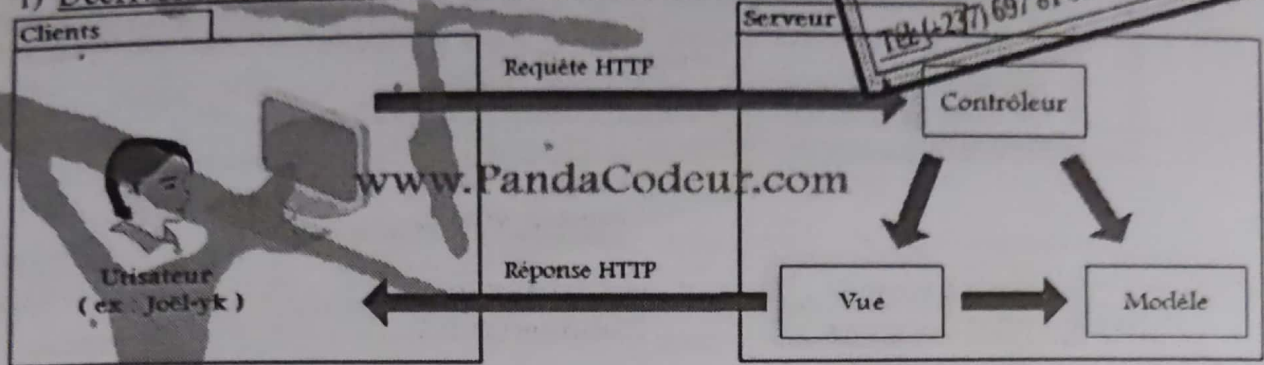
```
<%@ include file="myJSP.jsp" %>
```

Le Photocopieur THEO
L'homme des Bords
Tél (+237) 697 81 69 93 / 677 22 18 66

- 6) Explication La portée session : les objets possédant cette portée sont accessibles durant l'existence de la session en cours.

EXERCICE 2

1) Décrivons le modèle MVC avec java EE (SHEMA) :



Plus d'exercices corrigés sur www.PandaCodeur.com

* rôle de chacune des couches :

- le modèle : il gère tout ce qui concerne le traitement, le stockage, et la mise à jour des données de l'application.
- la vue : elle gère tout ce qui concerne l'interaction avec l'utilisateur et la présentation des données i.e. la mise en forme, l'affichage.
- le contrôleur : il gère tout ce qui concerne le contrôle des actions utilisateur et des données.

2) Etude comparative :

Le Photocopieur THEO
L'homme des Bords
Tél: (+237) 697 81 69 93 / 677 22 18 66

	ARCHITECTURE MVC	ARCHITECTURE CLIENT/SERVEUR
NATURE	Modèle de conception design pattern	Style d'architecture
COMMUNICATION	Communication 2 par 2 (triangulaire)	Communication entre les niveaux (bidirectionnelle)
DEPLOIEMENT (RESEAU)	Se déploie comme un seul processus dans un seul nœud du réseau	Se déploie comme 3 processus distincts sur 3 différents nœuds du réseau

Problème : Voir Examen Normale 2017-2018

Contact WhatsApp : +237 658395978 Réalisé par Joël_Yk

Le Photocopieur THEO
L'homme des Bords
Tél: (+237) 697 81 69 93 / 677 22 18 66

2020

by Joël JK

EXAMEN DE FIN DE SECOND SEMESTRE

Année Académique 2019 / 2020 - INF 332-Ingénierie des applications web - IN3 - Durée : 2 h

Exercice 1 (5 points)

- 1) Qu'est-ce qu'un serveur http ? Donnez en deux exemples. (1 pt)
- 2) A travers le protocole HTTP, le navigateur envoie des requêtes au serveur et le serveur lui renvoie des réponses :
 - a) Quel est le rôle du serveur HTTP (0,5 pt)
 - b) Quel est le rôle du navigateur ? (0,5 pt)
- 3) Quels sont les éléments constitutifs d'un serveur d'application ? (0,5 pt)
- 4) Qu'est-ce qu'un Framework ? Donnez deux exemples. (1 pt)
- 5) Qu'est-ce qu'un EDI ? Donnez la définition de ce sigle. (1 pt)
- 6) Donnez la structure des fichiers d'une application web JEE. (0,5 pt)

Exercice 2 : (5 points)

- 1) Quel est le rôle d'un serveur web ? et d'un serveur d'application ? (1 pt)
- 2) Quels sont les éléments logiciels constitutifs nécessaires à la réalisation et l'exécution d'une application web en technologie Java ? (1 pt)
- 3) Quelle est la différence entre les commandes GET et POST ? (1 pt)
- 4) Par quelle méthode de servlet est traitée une commande POST du protocole HTTP ? Donner sa signature. (1 pt)
- 5) De quel type est l'objet fourni par une servlet qui porte la réponse au client ? (0,5 pt)
- 6) De quel type est le flot de sortie sur lequel est écrit le code de la page HTML retournée au client ? (0,5 pt)

Problème (10 pts)

Soit le diagramme de classe de la figure 1 :

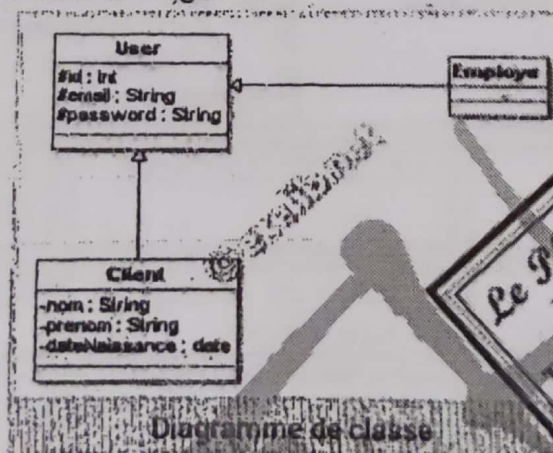


Figure 1: Diagramme de classe

PARTIE A : Couche accès aux données (4 points)

- 1) Créer les classes entités dans le package `ma.project.entity`.
- 2) Créer le fichier de configuration Hibernate dans le package `ma.project.config`.

- 3) Créer le fichier `HibernateUtil` permettant de créer une `SessionFactory` dans le package `ma.project.util`.
- 4) Créer l'interface `IDao` dans le package `ma.project.dao`.

PARTIE B : Couche service (2 points)

- 1) Créer les classes `ClientService` et `EmployeService` dans le package `ma.project.service`.
- 2) Créer une classe de test pour tester les services.

PARTIE C : Couche Présentation (4 points)

- 1) Créer une page d'inscription d'un client. Le mot de passe doit être haché.
- 2) Créer une page d'authentification. Cette page contient :
 - Une zone de saisie de login qui aura automatiquement le focus dès l'ouverture de la page, et une zone de saisie de mot de passe pour identifier l'utilisateur.
 - Deux liens, le premier vers la page de récupération de mot de passe, et le deuxième vers la page d'inscription.
 - Un texte par défaut dans les champs du formulaire d'authentification. Ces valeurs s'effacent dès que l'utilisateur active ces champs.
 - Une fois le visiteur identifié, un message de bienvenue, la date et l'heure du système s'affichent. Chaque visiteur sera redirigé vers son espace (Espace client, espace employé).
- 3) Au clic sur le lien mot de passe oublié, le formulaire de la figure 2 s'affiche :

Figure 2: demande du mail pour le code d'activation

- L'utilisateur doit saisir son mail, puis le système vérifie l'existence de l'utilisateur et envoie un code généré aléatoirement vers sa boîte email s'il existe, dans le cas contraire, le message Email n'existe pas sera affiché.
- Après envoi de l'email, l'utilisateur sera redirigé vers une page permettant de saisir le code envoyé (voir figure 3) :

Figure 3: Saisi du code de validation

- Si le code est correct, le système demande de saisir le nouveau mot de passe et la confirmation du mot de passe (une vérification côté client est obligatoire : les deux mots de passe doivent être identiques).



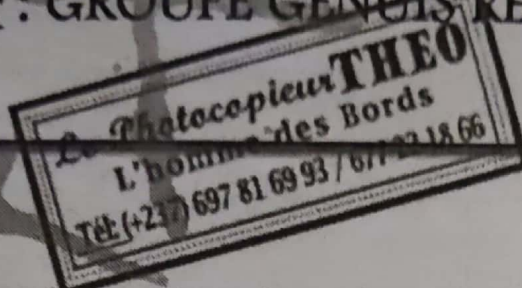
CORRECTION EXAMEN DE FIN DE SECOND SEMESTRE

(NORMALE 2019-2020)

INGENIERIE DES APPLICATION WEB

Proposez par : GROUPE GENUIS REPETITION

Par : Joël_yk



Exercice 1 :

- 1) Un serveur http : permet la gestion des requêtes et réponses http suivant l'architecture client- serveur.

Exemple : Nginx, Apache

- 2) a- rôle du serveur http : Le serveur http reçoit la requête, l'interprète et génère alors une page web qu'il enverra au client par le biais d'une réponse http.

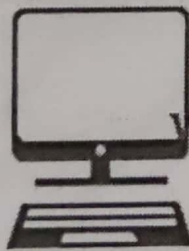
b- Rôle du navigateur :

- Il envoie tout d'abord une requête http au serveur pour lui demander la page correspondante.
- Il reçoit la réponse du serveur, via cette réponse il affiche la page web finale à l'utilisateur.

1-Le client saisit une URL



Client



2-Le navigateur envoie une requête HTTP au serveur

www.PandaCodeur.com



Serveur

3-Le serveur traite la requête HTTP et génère la page web demandée

4-Le serveur envoie une réponse HTTP au client

- 3) Les éléments consécutifs d'un serveur d'application :

- Le serveur HTTP ;
- Conteneur web ou conteneur de servlet.

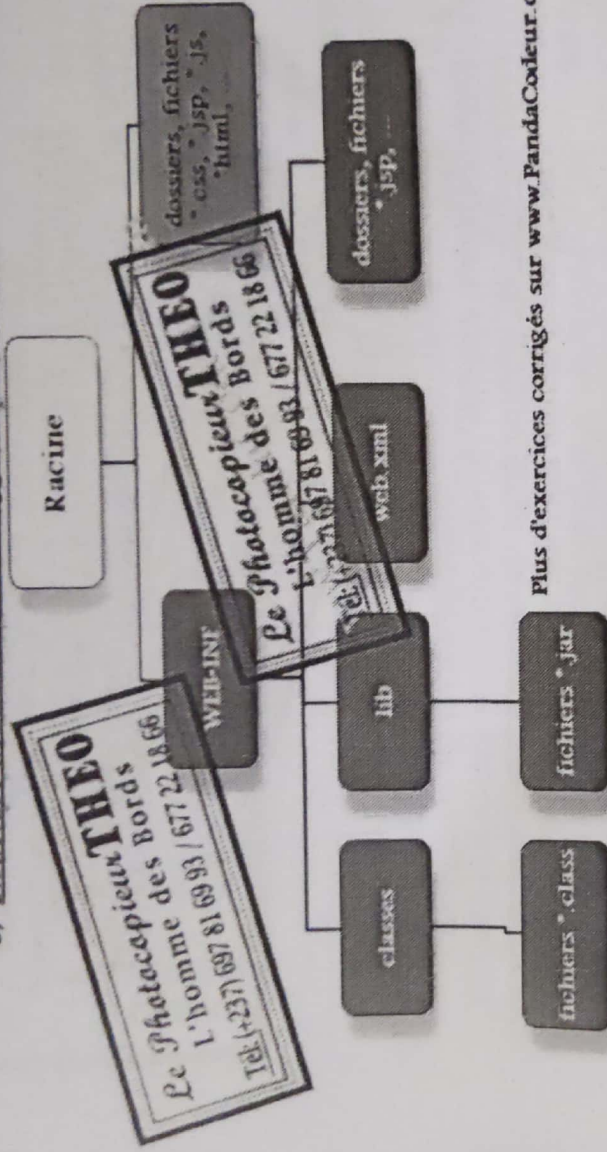
- 4) Un Framework : est un ensemble de composant qui servent à créer l'architecture et les grandes lignes de notre application.

Exemple : Spring, Hibernate, Jsf (Java Server Faces), Apache Wicket, Vert.X, Grails, Play, Google Web Kit (GWT), Apache Struts, Dropwizard, Spark | si tu désires connaître les avantages ou désavantages de chacun d'entre eux : www.Pandacodeur.com

5) Un EDI : est un ensemble d'outils qui permet d'augmenter la productivité des programmeurs qui développent des logiciels.

EDI= Environnement Développement Intègre.

6) Structure des fichiers d'une application web JEE



Plus d'exercices corrigés sur www.PandaCodeur.com

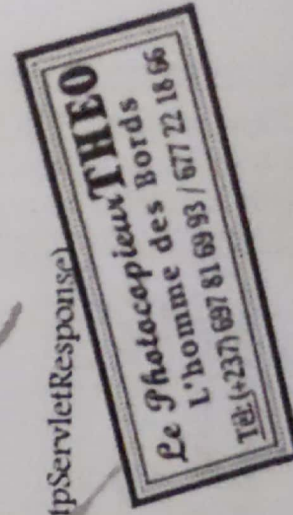
EXERCICE 2

- 1) Le rôle d'un Serveur Web est de pouvoir traiter une requête Http.
Le rôle d'un Serveur d'Application est de fournir une infrastructure de services pour l'exécution d'application.
- 2) Les Éléments Logiciels Constitutifs nécessaires à la réalisation et l'exécution d'une application web en technologie Java :
 - Le Serveur Http
 - Le Serveur d'Application
 - Le Conteneur JSP
 - Le Conteneur de Servlet
 - La Plateforme JEE.
- 3) Différence entre les commandes GET et POST : la différence fondamentale entre les méthodes GET et POST est que selon un critère de bonne pratique la méthode POST doit être utilisée pour réaliser les opérations qui ont un effet sur la ressource, alors que la méthode GET est la méthode utilisée par le client pour récupérer une ressource web au serveur via une url.
- 4) Elle est traitée par la méthode : doPost ()

Sa signature :

doPost (HttpServletRequest , HttpServletResponse)

- 5) Cet objet est de type : HttpServletRequest
- 6) Le Type de Flot de sortie est : PrintWriter



Problème 10 pts :

PARTIE A : Couche Accès aux Données

1) Creation des Classes Entites dans le Package **ma.project.entity**

Concernant le Framework Hibernate petit detour rapide : www.Pandacodeur.com

a- Entité : Client

```
package ma.project.entity;
import java.util.Date;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Temporal;
import javax.persistence.TemporalType;
@Entity
public class Client extends User(
    @GeneratedValue
    private String nom;
    private String prenom;
    @Temporal(TemporalType.DATE)
    private Date datenaissance;

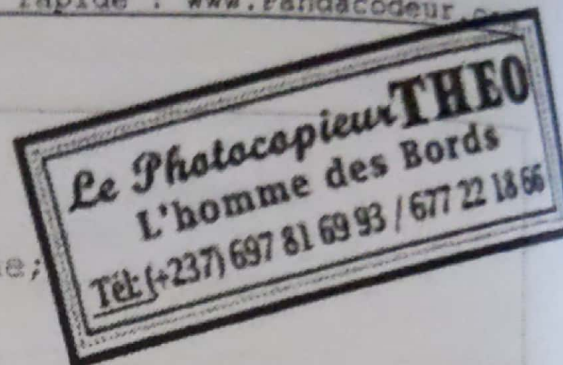
    public Client() {
    }

    public Client(String nom, String prenom, Date
    dateNaissance, String email, String password, int etat, String
    code) {
        super(email, password, etat, code);
        this.nom = nom;
        this.prenom = prenom;
        this.datenaissance = dateNaissance;
        this.etat = etat;
        this.code = code;
    }

    public String getNom()
        return nom;
    }

    public void setNom(String nom) {
        this.nom = nom;
    }

    public String getPrenom() {
        return prenom;
    }
```




```

    }

    public void setPrenom(String prenom) {
        this.prenom = prenom;
    }

```

```

    public Date getDatenaissance() {
        return datenaissance;
    }

```

```

    public void setDatenaissance(Date datenaissance) {
        this.datenaissance = datenaissance;
    }

```

b- Entité : Employer

```

package ma.projet.entity;
import javax.persistence.Entity;
@Entity
public class Employer extends User {
    public Employer() {
    }
}

```

1) c- Entité : User

```

package ma.projet.entity;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;
import javax.persistence.Inheritance;
import javax.persistence.InheritanceType;
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
public class User {
    @Id
    @GeneratedValue
    protected int id;
    @Column(unique = true)
    protected String email;
    protected String password;
    protected int etat;
    protected String code;

    public User() {
    }
}

```

Le Photocopieur THEO
L'homme des Bords
Tél: (+237) 697 81 69 93 / 677 22 18 66

Le Photocopieur THEO
L'homme des Bords
Tél: (+237) 697 81 69 93 / 677 22 18 66

Le Photocopieur THEO
L'homme des Bords
Tél: (+237) 697 81 69 93 / 677 22 18 66

```

    }

    public User(String email, String password, int etat,
String code) {
        this.email = email;
        this.password = password;
        this.etat = etat;
        this.code = code;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public int getEtat() {
        return etat;
    }

    public void setEtat(int etat) {
        this.etat = etat;
    }

    public String getCode() {
        return code;
    }

    public void setCode(String code) {
        this.code = code;
    }
}

```



2-Créer le fichier de Configuration HIBERNATE

```

<!DOCTYPE hibernate-configuration PUBLIC "-
//Hibernate/Hibernate Configuration DTD 3.0//EN"

```



```

"http://hibernate.sourceforge.net/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</
property>
    <property
name="hibernate.connection.driver_class">com.mysql.jdbc.Drive
r</property>
    <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/s
essiontp?zeroDateTimeBehavior=convertToNull</property>
    <property
name="hibernate.connection.username">root</property>
    <property name="hibernate.hbm2ddl.auto">update</property>
    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.format_sql">true</property>
    <mapping class="ma.projet.entity.User"/>
    <mapping class="ma.projet.entity.Client"/>
    <mapping class="ma.projet.entity.Employer"/>
  </session-factory>
</hibernate-configuration>
  )
)

```



3) Création du Fichier HibernateUtil

```

package ma.projet.util;

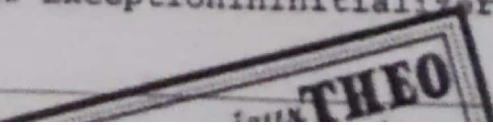
import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.SessionFactory;

public class HibernateUtil {

    private static final SessionFactory sessionFactory;

    static {
        try {
            // Create the SessionFactory from standard
(hibernate.cfg.xml)
            // config file.
            sessionFactory = new
AnnotationConfiguration().configure().buildSessionFactory();
        } catch (Throwable ex) {
            // Log the exception.
            System.err.println("Initial SessionFactory
creation failed." + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }
}

```



```
public static SessionFactory getSessionFactory() {
    return sessionFactory; } }
```

4) Création Interface IDao :

```
package IDao;

import java.util.List;
public interface IDao<T> {

    boolean create(T o);
    boolean delete(T o);
    boolean update(T o);
    T getById(int id);
    List<T> getAll();
}
```



PARTIE B : Couche Service

1) Création des Classes :

a- ClientService

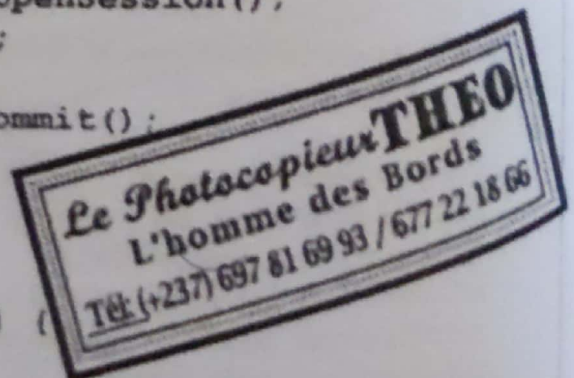
```
package ma.projet.service;

import java.util.List;
import ma.projet.util.HibernateUtil;
import ma.projet.entity.Client;
import org.hibernate.Session;

public class ClientService implements IDao.IDao<Client> {

    @Override
    public boolean create(Client o) {
        Session session =
        HibernateUtil.getSessionFactory().openSession();
        session.beginTransaction();
        session.save(o);
        session.getTransaction().commit();
        return true;
    }

    @Override
    public boolean delete(Client o) {
        Session session =
        HibernateUtil.getSessionFactory().openSession();
```




```

        session.beginTransaction();
        session.delete(o);
        session.getTransaction().commit();
        return true;
    }

```



```

    @Override
    public boolean update(Client o) {
        Session session =
        HibernateUtil.getSessionFactory().openSession();
        session.beginTransaction();
        session.update(o);
        session.getTransaction().commit();
        return true;
    }

```

```

    @Override
    public Client getById(int id) {
        Client client = null;
        Session session =
        HibernateUtil.getSessionFactory().openSession();
        session.beginTransaction();
        client = (Client) session.get(Client.class, id);
        session.getTransaction().commit();
        return client;
    }

```

```

    @Override
    public List<Client> getAll() {
        List<Client> clients = null;
        Session session =
        HibernateUtil.getSessionFactory().openSession();
        session.beginTransaction();
        clients = session.createQuery("from Client").list();
        session.getTransaction().commit();
        return clients;
    }

```



```

    public Client getByEmail(String email) {
        Client c = null;
        Session session =
        HibernateUtil.getSessionFactory().openSession();
        session.beginTransaction();
        c = (Client) session.createQuery("from Client where
        email = ?").setParameter(0, email).uniqueResult();
        session.getTransaction().commit();
        return c;
    }
}

```

b- EmployeeService

```
package ma.projet.service;
```

```
import java.util.List;  
import ma.projet.util.HibernateUtil;  
import ma.projet.entity.Employer;  
import org.hibernate.Session;
```

```
public class EmployeeService implements IDao.IDao<Employer> {
```

```
    @Override
```

```
    public boolean create(Employer o) {
```

```
        Session session =  
        HibernateUtil.getSessionFactory().openSession();  
        session.beginTransaction();  
        session.save(o);  
        session.getTransaction().commit();  
        return true;
```

```
    }
```

```
    @Override
```

```
    public boolean delete(Employer o) {
```

```
        Session session =  
        HibernateUtil.getSessionFactory().openSession();  
        session.beginTransaction();  
        session.update(o);  
        session.getTransaction().commit();  
        return true;
```

```
    }
```

```
    @Override
```

```
    public boolean update(Employer o) {
```

```
        Session session =  
        HibernateUtil.getSessionFactory().openSession();  
        session.beginTransaction();  
        session.delete(o);  
        session.getTransaction().commit();  
        return true;
```

```
    }
```

```
    @Override
```

```
    public Employer getById(int id)
```

```
        Employer employer = null;  
        Session session =  
        HibernateUtil.getSessionFactory().openSession();  
        session.beginTransaction();  
        employer = (Employer) session.get(Employer.class,  
        id);  
        session.getTransaction().commit();  
        return employer;
```

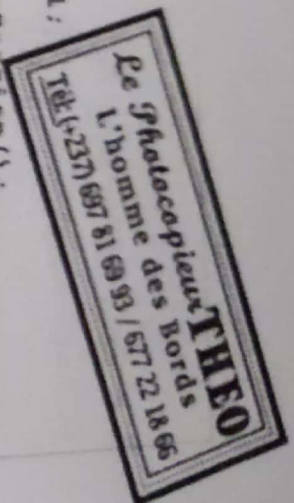
Le Photocopieur THEO
L'homme des Bords
Tél: (+237) 697 81 69 93 / 677 22 18 66

Le Photocopieur THEO
L'homme des Bords
Tél: (+237) 697 81 69 93 / 677 22 18 66


```

    )
    @Override
    public List<Employer> getAll() {
        List<Employer> employees = null;
        Session session =
            HibernateUtil.getSessionFactory().openSession();
        session.beginTransaction();
        employees = session.createQuery("from
            Employer").list();
        session.getTransaction().commit();
        return employees;
    }
}

```



2) Classe de Test !

```

package ma.projet.test;

import ma.projet.entity.Employer;
import ma.projet.service.ServiceEmployer;
import ma.projet.util.HibernateUtil;

```

```

public class Test {
    public static void main(String[] args) {
        HibernateUtil.getSessionFactory().openSession();
    }
}

```

PARTIE C : Couche Présentation

1) Création Page D'inscription d'un Client :

```

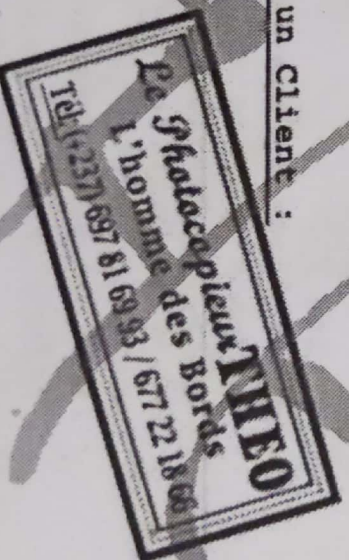
package ma.projet.controlleur;

```

```

import java.io.IOException;
import java.io.PrintWriter;
import java.util.Date;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import ma.projet.entity.Client;
import ma.projet.service.ClientService;
import ma.projet.service.SendMail;
import ma.projet.util.Util;

```



```

@WebServlet(name = "Inscription", urlPatterns =
{"/Inscription"})
public class Inscription extends HttpServlet {

    protected void processRequest(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        String nom = request.getParameter("nom");
        String prenom = request.getParameter("prenom");
        String email = request.getParameter("email");
        String date = request.getParameter("date").replace("-",
            "/");
        String password =
            Util.mds(request.getParameter("password"));
        ClientService cs = new ClientService();
        cs.create(new Client(nom, prenom, new Date(date),
            email, password, 0, null));

        response.sendRedirect("Authentication.jsp?email="+email);
    }

    // <editor-fold defaultstate="collapsed"
    desc="HttpServlet methods. Click on the + sign on the left to
    edit the code.">
    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        processRequest(request, response);
    }

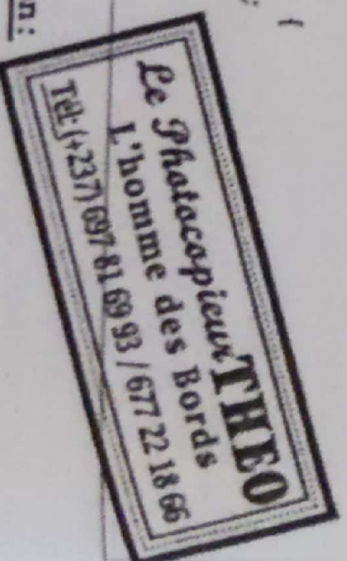
    @Override
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    public String getServletInfo() {
        return "Short description";
    } // </editor-fold>
}

```

2) Création Page D'authentification:

Package ma.projet.controleur;



Le Photocopieur **THEO**
L'homme des Bords
Tél (+237) 697 81 69 93 / 677 22 18 66

Le Photocopieur **THEO**
L'homme des Bords
Tél (+237) 697 81 69 93 / 677 22 18 66

Le Photocopieur **THEO**
L'homme des Bords
Tél (+237) 697 81 69 93 / 677 22 18 66

Le Photocopieur **THEO**
L'homme des Bords
Tél (+237) 697 81 69 93 / 677 22 18 66

Le Photocopieur **THEO**
L'homme des Bords
Tél (+237) 697 81 69 93 / 677 22 18 66

Photocepteur THEO
L'homme des Bords
Tél (+237) 687 81 69 93 / 677 22 18 66

Photocepteur THEO
L'homme des Bords
Tél (+237) 687 81 69 93 / 677 22 18 66

Photocepteur THEO
L'homme des Bords
Tél (+237) 687 81 69 93 / 677 22 18 66

Photocepteur THEO
L'homme des Bords
Tél (+237) 687 81 69 93 / 677 22 18 66

Photocepteur THEO
L'homme des Bords
Tél (+237) 687 81 69 93 / 677 22 18 66

Photocepteur THEO
L'homme des Bords
Tél (+237) 687 81 69 93 / 677 22 18 66

Photocepteur THEO
L'homme des Bords
Tél (+237) 687 81 69 93 / 677 22 18 66

Photocepteur THEO
L'homme des Bords
Tél (+237) 687 81 69 93 / 677 22 18 66

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.xml.soap.SOAPException;
import javax.xml.soap.SOAPMessage;
import javax.xml.ws.EndpointResolver;
import javax.xml.ws.Resolver;
import org.springframework.web.servlet.mvc.annotation.AnnotationMethodHandlerMethod;
import org.springframework.web.servlet.mvc.annotation.AnnotationMethodHandlerMethod;
```

```
@WebServlet(name = "Verifier", urlPatterns = {"/Verifier"})
public class Verifier extends HttpServlet {
```

```
protected void processRequest(HttpServletRequest request,  
    HttpServletResponse response)
```

```
throws ServletException, IOException {
    response.setContentType("text/html;charset=UTF-8");
    int code =
        parseInt(request.getParameter("code"));
    ClientService cl = new ClientService ();
    HttpSession session = request.getSession();
    Client c = (Client) session.getAttribute("client");
    if (c != null) {
```

```
if (Integer.parseInt(c.getCode())==code) {
```

```
response.sendRedirect("updatemotdepasse.jsp");
```

```

    } else {
        response.sendRedirect("virfier.jsp?msg= le
code est incorrect Zutte!! ");
    }
}

```

```

    } else {
        response.sendRedirect("virfier.jsp?msg= session
        vide!!");
    }
}

```

```
// <editor-fold defaultstate="collapsed"
desc="HttpServlet methods. Click on the + sign on the left to
edit the code.">
```

```

@Override
protected void doGet(HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response)
}

```

```
@Override
protected void doPost(HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {
```

Scanné avec CamScanner


```

        processRequest(request, response);
    }

```

```

@Override
public String getServletInfo() {
    return "Short description";
} // </editor-fold>

```

b) update

```

package ma.projet.controlleur;

```

```

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import ma.projet.entity.Client;
import ma.projet.service.ClientService;
import ma.projet.util.Util;

```

```

@WebServlet(name = "updatemotdepasse", urlPatterns =
{"/updatemotdepasse"})

```

```

public class updatemotdepasse extends HttpServlet {
    protected void processRequest(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {

```

```

        String password=request.getParameter("password");
        String passwordcnf=request.getParameter("passwordcnf");
        ClientService cl = new ClientService();
        if(password.equals(passwordcnf)) {
            HttpSession session = request.getSession();
            Client c = (Client) session.getAttribute("client");
            c.setPassword(Util.md5(password));
            cl.update(c);

```

```

            response.sendRedirect("Authentication.jsp?email="+c.getEmail());
        }
    }

```

```

    }

```

```

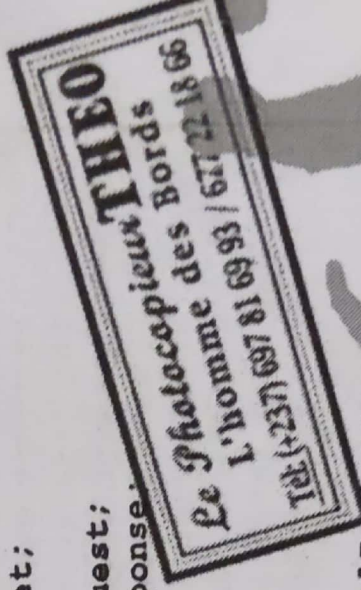
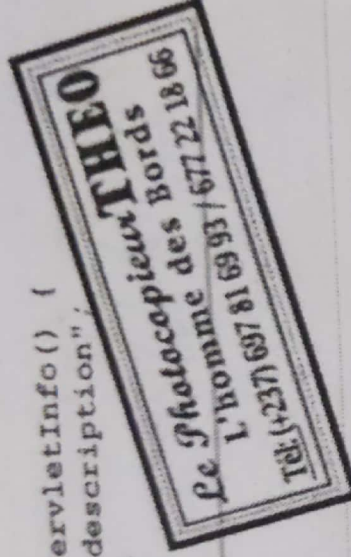
    response.sendRedirect("updatemotdepasse.jsp?email=mot
de passe incorrect");
}

```

```

}

```



```
// <editor-fold defaultstate="collapsed"
desc="HttpServletRequest methods. Click on the + sign on the left to
edit the code.">
```

```
@Override
protected void doGet(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {
    processRequest(request, response);
}
```

```
@Override
protected void doPost(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {
    processRequest(request, response);
}
```

```
@Override
public String getServletInfo() {
    return "Short description";
} // </editor-fold>
```

Le Photocopieur THEO
L'homme des Bords
Tél: (+237) 697 81 69 93 / 677 22 18 66

Le Photocopieur THEO
L'homme des Bords
Tél: (+237) 697 81 69 93 / 677 22 18 66

Contact WhatsApp : +237 658395978

Réaliser Par Joël_Yk

Exercice 1 (5 points)

- 1) Quelle différence y a-t-il entre JAVA EE et JAVA SE ? (1 pt)
- 2) Qu'est-ce qu'un EDI ? Donnez la définition de ce sigle. (1 pt)
- 3) Donnez quatre fonctions principales d'un serveur d'application. (2 pts)
- 4) Donnez deux serveurs d'applications qui sont des solutions propriétaires et payantes, et deux autres qui sont des solutions libres et gratuites (1 pt)

Exercice 1 : (5 points)

- 1) Quelle est la différence entre les commandes GET et POST (1 pt)
- 2) Par quelle méthode de servlet est traitée une commande GET du protocole HTTP ? Donner sa signature. (1 pt)
- 3) De quel type est le flot de sortie sur lequel est écrit le code de la page HTML retournée au client ? (1 pt)
- 4) A quoi sert le fichier web.xml ? Quelles informations y trouve-t-on ? (2 pts)

Exercice 3 : (5 points)

Ecrire la méthode doGet d'une servlet qui affiche sur une page les informations suivantes :

- nom du serveur
- valeurs associées aux clés connection et keep-alive de l'en-tête
- adresse IP du client

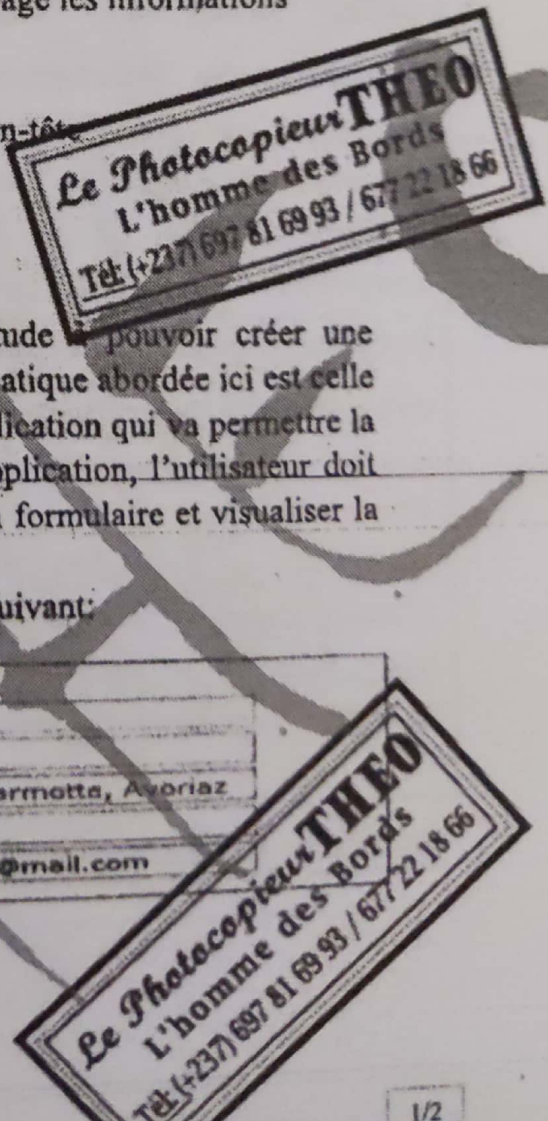
Problème (5 pts)

L'objectif de ce problème est de vérifier votre aptitude à pouvoir créer une application web en utilisant la technologie Java EE. La thématique abordée ici est celle du commerce en ligne. Vous allez créer un embryon d'application qui va permettre la création et la visualisation d'un client. A travers votre application, l'utilisateur doit pouvoir créer un client en saisissant des données depuis un formulaire et visualiser la fiche client en résultant.

La création d'un client se fait en remplissant le formulaire suivant:

Informations client	
Nom *	DUPOND
Prénom	Marcel
Adresse de livraison *	12 rue de la marmotte, Avoriaz
Numéro de téléphone *	0412345678
Adresse email	marcel.dupond@mail.com

En cas de succès, la page suivante s'affiche à l'utilisateur:



EXAMEN DE RATTRAPAGE

Année Académique 2019 / 2020 - INF 332-Ingénierie des applications web - IN3 - Durée : 2 h

Nom : DUPOND

Prénom : Marcel

Adresse : 12 rue de la marmotte, Avoriaz

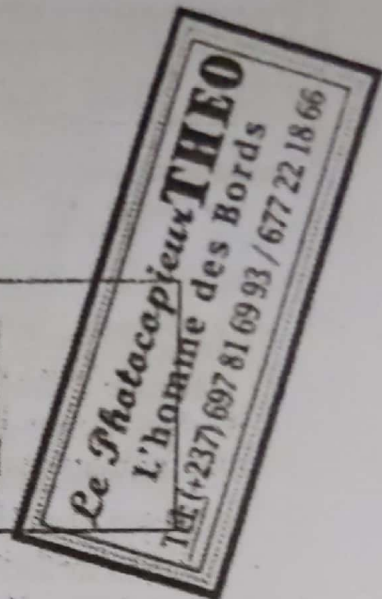
Numéro de téléphone : 0412345678

Email : marcel.dupond@mail.com

Et en cas d'échec (Exemple d'oubli d'un champ obligatoire):

Informations client

Nom *	DUPOND
Prénom	Marcel
Adresse de livraison *	12 rue de la marmotte, Avoriaz
Numéro de téléphone *	
Adresse email	marcel.dupond@mail.com



On suppose que le projet Web créé à cet effet est nommé TPIN3, que le formulaire de saisi des informations du client est dans le fichier `creerClient.jsp` et qu'il peut être accessible à tout le monde. La page qui affiche la confirmation de la création d'un client ne doit pas pouvoir être accessible directement à partir d'une URL du navigateur, elle doit se nommer `confirmClient.jsp`. L'envoi des requêtes du formulaire se fait par la méthode Post.

- 1) Sachant que la servlet `Client.java` devra être accessible via l'url `/client`, donner le contenu qu'il faudra ajouter au fichier `web.xml` pour effectuer le mapping entre cette url et la servlet. La servlet devra se trouver dans le package `com.tp.in3`. (1 pt)
- 2) Créer le formulaire `creerClient.jsp`. (1 pt)
- 3) Créer le bean représentant les données récupérées depuis le formulaire `creerClient.jsp`. (1 pt)
- 4) Créer la servlet `Client.java` chargée de récupérer les données envoyées par le formulaire, de les enregistrer dans le bean et de les transférer à `confirmClient.jsp`. (1 pt)
- 5) Créer la JSP `confirmClient.jsp` chargée de récupérer les données envoyées depuis la servlet `Client.java`. (1 pt)

Servlet : client
fichier : client

CORRECTION EXAMEN DE FIN DE SECOND SEMESTRE
(RATTRAPAGE 2019-2020)
INGENIERIE DES APPLICATION WEB

Proposez par : GROUPE GENUIS REPETITION

Par : Joël yk

Le Photocopieur THEO
L'homme des Bords
Tél (+237) 697 81 69 93 / 677 22 18 66

Exercice I :

1) La Différence :

Le Java Entreprise Edition (Java EE) comme son nom l'indique a été crée pour le développement d'application d'entreprise. Il est destiné au développement d'application web. Alors que le Java Standard Edition (Java SE) est destiné à la création d'application bureautique (App de Bureau). Le Java EE fournit un ensemble d'extension au Java Standard afin de faciliter la création d'application centralisées.

2) Un EDI : est un ensemble d'outils qui permet d'augmenter la productivité des programmeurs qui développent des logiciels.

EDI= Environnement Développement Intègre.

3) 04 fonctions principales d'un serveur d'application :

- Récupération des requêtes Http issues du client.
- les mettre dans des boites, des objets, que notre code sera capable de manipuler.
- faire passer les objets dans la moulinette qu'est notre application via le conteneur.
- renvoyer les réponses Http aux clients en se basant sur les objets retournes par notre code.

4) -02 serveurs d'application qui sont les solutions payantes : WebSphere As, WebLogic, Oracle Application Server, Sun Java System Application Server.

-02 serveurs d'application qui sont des solutions libres et gratuites : Jboss, Apache Tomcat, JOnAS, GlassFish, Geronimo, OpenEJB, JFox, JRun.

EXERCICE 2

- 1) **Différence entre les commandes GET et POST :** la différence fondamentale entre les méthodes GET et POST est que selon un critère de bonne pratique la méthode POST doit être utilisée pour réaliser les opérations qui ont un effet sur la ressource, alors que la méthode GET est la méthode utilise par le client pour récupérer une ressource web au serveur via une url.

2) Elle est traite par la méthode : doGet ()

Sa signature :

doGet (HttpServletRequest , HttpServletResponse)

Le Photocopieur THEO
L'homme des Bords
Tél (+237) 697 81 69 93 / 677 22 18 66

- 3) Le Type de Flot de sortie est : **PrintWriter**
4) Le Fichier **web.xml** permet de déployer l'application sur une plateforme d'exécution.

Les Informations qu'il y si trouve sont :

- Les Spécifications des Servlets (nom et URL)
- Le Contexte pour l'Application (définition)
- Les pages d'erreur (définition)
- Les Caractéristiques de Session

EXERCICE 3

Ecrivons une méthode **doGet** d'une servlet qui affiche sur une page les informations suivantes :

- nom du serveur
- valeur associées aux clés connection et Keep-alive de l'entête
- adresse IP du client.

```
import javax.servlet.* ;
import javax.servlet.http.* ;
import javax.io.* ;

Public class InfoServeur extends HttpServlet {

Public void doGet ( HttpServletRequest request,
HttpServletResponse response) throws ServletException ,
IOException {

response.setContentType ("text/plain")
PrintWriter out = response.getWriter ();

Out.println("Serveur: " + request.getServerName());

Out.println ("Header connection:" + request.getHeader
("connection") );

Out.println ( " Header keep.alive :" + request.getHeader
("keep-alive " ) );

Out.println (" adresse IP du client : adresse IP du client : "
+ request.getRemoteAddr( ) ) ;
}
```

Problème : Voir Examen Normale 2016-2017

Contact WhatsApp : +237 658395978

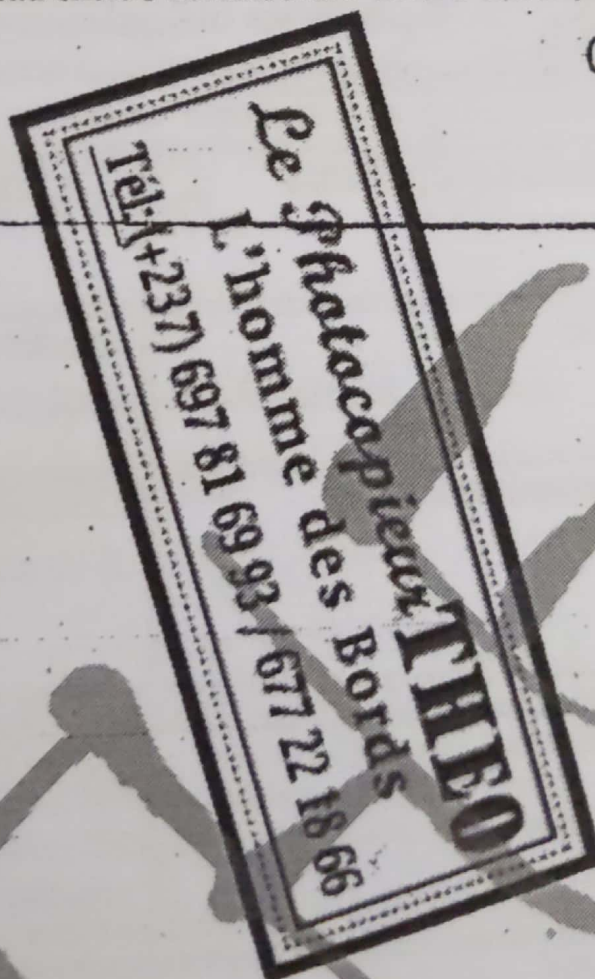


CONTROLE CONTINU

Année Académique 2018 / 2019 - INF 20-Application Internet - IN3 - Durée : 1 h

Exercice 1 (10 points)

- 1) Quelles différences faites-vous entre un serveur http et un serveur d'application ? (1 pt)
- 2) Donnez quatre fonctions principales d'un serveur d'application. (2 pts)
- 3) Donnez deux serveurs d'applications qui sont des solutions propriétaires et payantes, et deux autres qui sont des solutions libres et gratuites (2 pts)
- 4) Qu'est-ce qu'un design pattern ? Donnez en un exemple. (1 pt)
- 5) Décrire le modèle MVC avec java EE par un schéma et donner le rôle de chacune des couches qui le constitue. (2 pts)
- 6) Faire une étude comparative sous forme de tableau entre l'architecture Client-Serveur et l'architecture MVC. (2 pts)

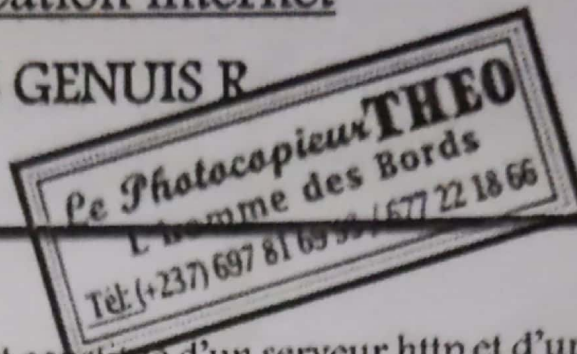


Correction du contrôle continu 2018-2019

application internet

Proposez par : GROUPE GENUIS B

Par : Joël_yk

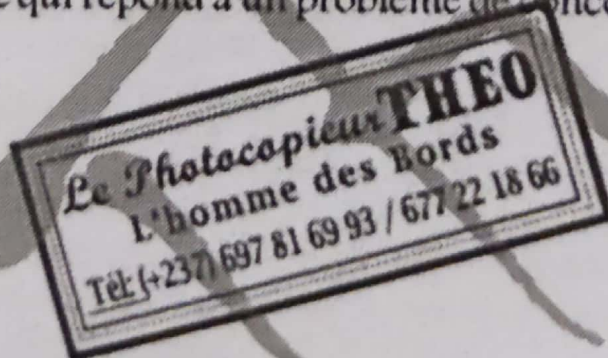


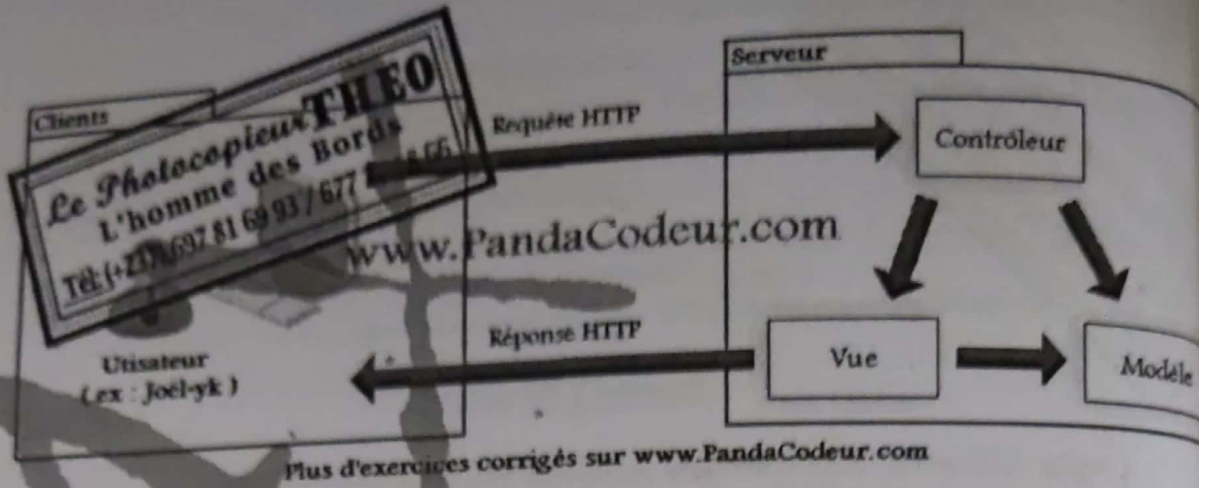
Exercice 1

1. Un serveur d'application est constitué d'un serveur http et d'un conteneur web (exemple : TOM cat) tandis qu'un Serveur http permet la gestion des requêtes et réponses http.
2. 04 fonctions principales d'un serveur d'application :
 - Récupérer les requêtes Http issues du client.
 - Les mettre dans les boîtes, des objets que notre code sera capable de manipuler.
 - Faire passer ces objets dans la moulinette qu'est notre application via le conteneur.
 - Renvoyer les réponses Http aux clients en se basant sur les objets retournés par notre code.
3. 02 serveurs d'applications qui sont des solutions payantes :
 - WebSphere AS
 - Web Logic
 - Oracle Application Server

02 serveurs d'application qui sont des solutions libres et gratuites :

 - J Boss, Apache Tom cat, JOnAS, GlassFish, Geronimo, OpenEJB, JFox, JRun.
4. Un design pattern au modèle de conception ou patron de conception : est une simple bonne pratique qui répond à un problème de conception d'une application.
Exemple : le modèle MCV
5. Schéma du modèle MCV :





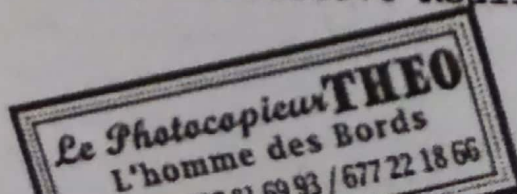
Rôle de chacune des couches qui le constitue :

- Le modèle : gère tout ce qui concerne le traitement, le stockage et la mise à jour des données de l'application.
- La vue : gère tout ce qui concerne l'interaction avec l'utilisateur et la présentation des données, i.e. la mise à jour et l'affichage.
- Le contrôleur gère tout ce qui concerne le contrôle des actions utilisateurs et les données.

6. Etude comparatives

Eléments comparatifs	Architecture MCV	Architecture client/server
Nature	modèle de conception, design pattern	Style d'architecture
Communication	Communication 2 par 2 (triangulaire)	Communication entre les niveaux (bidirectionnelle)
Déploiement sur le réseau	se déploie comme un seul processus dans un seul nœud du réseau	Se déploie comme 3 processus distincts sur différents nœuds du réseau
Objectif	Séparation des tâches, spécialisation	Centralisation des ressources, évolutivité, grande

Contact WhatsApp : +237 658395978 Réaliser Par Joël_Yk



Exercice 1 (5 points)

- 1) Qu'est-ce qu'un serveur http ? Donnez en deux exemples. (1 pt)
- 2) A travers le protocole HTTP, le navigateur envoie des requêtes au serveur et le serveur lui renvoie des réponses :
 - a) Quel est le rôle du serveur HTTP ? (0,5 pt)
 - b) Quel est le rôle du navigateur ? (0,5 pt)
- 3) Quels sont les éléments constitutifs d'un serveur d'application ? (0,5 pt)
- 4) Qu'est-ce qu'un Framework ? Donnez deux exemples. (1 pt)
- 5) Qu'est-ce qu'un EDI ? Donnez la définition de ce sigle. (1 pt)
- 6) Donnez la structure des fichiers d'une application web JEE. (0,5 pt)

Exercice 2 : (5 points)

- 1) Quelle est la différence entre les commandes GET et POST (1 pt)
- 2) Par quelle méthode de servlet est traitée une commande GET du protocole HTTP ? Donner sa signature. (1 pt)
- 3) De quel type est l'objet fourni par une servlet qui porte la réponse au client ? (1 pt)
- 4) Quelles balises doivent-elles être introduites dans un fichier web.xml pour qu'une servlet (contenue dans le fichier Hello.java du package servlets) soit initialisée avec des paramètres ayant respectivement les valeurs "2009" et "world"? (2 pts)

Exercice 3 : (4 points)

Ecrire la méthode doGet d'une servlet qui affiche sur une page les informations suivantes :

- nom du serveur
- valeurs associées aux clés connection et keep-alive de l'en-tête
- adresse IP du client

Problème (6 pts)

L'objectif de ce problème est de vérifier votre aptitude à pouvoir créer une application web en utilisant la technologie Java EE. La thématique abordée ici est celle du commerce en ligne. Vous allez créer un embryon d'application qui va permettre la création et la visualisation de commandes. A travers notre application, l'utilisateur doit pouvoir créer une commande en saisissant des données depuis un formulaire et visualiser la fiche commande en résultant.

La création d'une commande se fait en remplissant le formulaire suivant :

Informations commande

- Date *
- Montant *
- Mode de paiement *
- Statut du paiement
- Mode de livraison *
- Statut de la livraison

499.90
Chèque
45H chrono

En cas de succès, la page suivante s'affiche à l'utilisateur :

Date : 14/06/2012 10:37:16

Montant : 499.9

Mode de paiement : Chèque

Statut du paiement :

Mode de livraison : 48H chrono

Statut de la livraison :

Et en cas d'échec (Saisie d'un montant erroné par exemple) :

Informations commande

Date *

Montant *

Mode de paiement *

Statut du paiement

Mode de livraison *

Statut de la livraison

abcdef

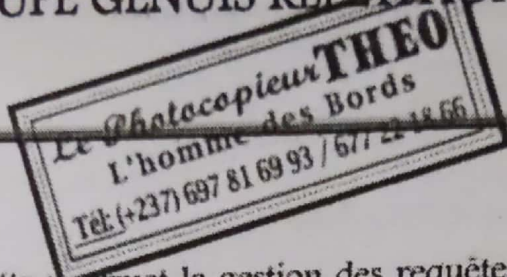
Chèque

On suppose que le projet Web créé à cet effet est nommé TPIN3, que le formulaire de saisie des informations du client est dans le fichier `creerCommande.jsp` et qu'il peut être accessible à tout le monde. La page qui affiche la confirmation de la création d'un client ne doit pas pouvoir être accessible directement à partir d'une URL du navigateur, elle doit se nommer `confirmCommande.jsp`. L'envoi des requêtes du formulaire se fait par la méthode Post.

- 1) Créer le formulaire `creerCommande.jsp`. (1.5 pt)
- 2) Créer le bean représentant les données récupérées depuis le formulaire `creerCommande.jsp`. (1.5 pt)
- 3) Créer la servlet `Commande.java` chargée de récupérer les données envoyées par le formulaire, de les enregistrer dans le bean et de les transférer à `confirmCommande.jsp`. (2 pts)
- 4) Créer la JSP `confirmCommande.jsp` chargée de récupérer les données envoyées depuis la servlet `Commande.java`. (1 pt)

CORRECTION EXAMEN DE FIN DE PREMIER SEMESTRE
(NORMALE 2018-2019)
APPLICATION INTERNET

Proposez par : GROUPE GENUIS REPETITION
Par : Joël_yk



Exercice 1 :

- 1) **Un serveur http** : permet la gestion des requêtes et réponses http suivant l'architecture client- serveur.

Exemple : Nginx, Apache

- 2) a- **rôle du serveur http** : Le serveur Http reçoit la requête, l'interprète et génère alors une page web qu'il enverra au client par le biais d'une réponse Http.

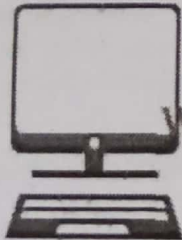
b- **Rôle du navigateur** :

- Il envoie tout d'abord une requête http au serveur pour lui demander la page correspondante.
- Il reçoit la réponse du serveur, via cette réponse il affiche la page web finale à l'utilisateur.

1-Le client saisit une URL



Client



2-Le navigateur envoie une requête HTTP au serveur

www.PandaCodeur.com



Serveur

3-Le serveur traite la requête HTTP et génère la page web demandée

4-Le serveur envoie une réponse HTTP au client

- 3) Les éléments consécutifs d'un serveur d'application :

- Le serveur HTTP ;
- Conteneur web ou conteneur de servlet.

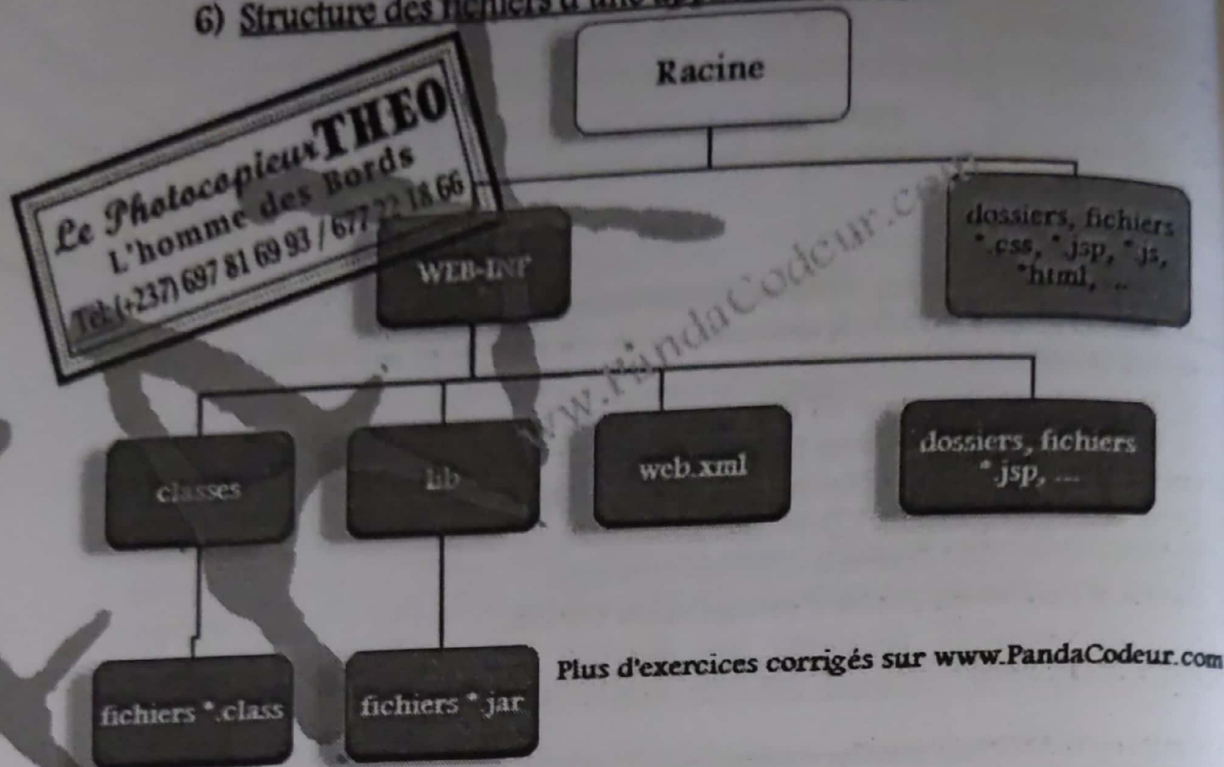
- 4) **Un Framework** : est un ensemble de composant qui servent à créer l'architecture et les grandes lignes de notre application.

Exemple : Spring, Hibernate ...

- 5) **Un EDI** : est un ensemble d'outils qui permet d'augmenter la productivité des programmeurs qui développent des logiciels.
EDI= Environnement Développement Intègre.



6) Structure des fichiers d'une application web JEE



EXERCICE 2

- 1) Différence entre les commandes GET et POST : la différence fondamentale entre les méthodes GET et POST est que selon un critère de bonne pratique la méthode POST doit être utilisée pour réaliser les opérations qui ont un effet sur la ressource, alors que la méthode GET est la méthode utilisée par le client pour récupérer une ressource web au serveur via une url.
- 2) Elle est traitée par la méthode : `doGet ()`
Sa signature :
`doGet (HttpServletRequest , HttpServletResponse)`
- 3) Cet objet est de type : `HttpServletResponse`
- 4) Balise :

<Servlet>

```

<Servlet-name> Hello </Servlet-name>
<Servlet-class> Servlets.Hello </Servlet-class>
<init-param>
  <param-name> Date </param-name>
  <param-value> 2009 </param-value>
</init-param>
<init-param>
  <param-name> Mot </param-name>
  <param-value> World </param-value>
</init-param>

```

</Servlet>

Le Photocopieur THEO
L'homme des Bords
Tél (+237) 697 81 69 93 / 677 22 18 66

EXERCICE 3

Ecrivons une méthode doGet d'une servlet qui affiche sur une page les informations suivantes :

- nom du serveur
- valeur associées aux clés connection et Keep-alive de l'entête
- adresse IP du client.

```
import javax.servlet.* ;
import javax.servlet.http.* ;
import javax.io.* ;

public class InfoServeur extends HttpServlet {

    public void doGet ( HttpServletRequest request,
        HttpServletResponse response) throws ServletException ,
        IOException {

        response.setContentType ("text/plain")

        PrintWriter out = response.getWriter ();

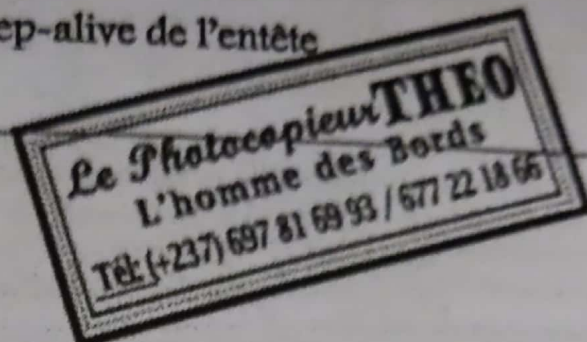
        Out.println("Serveur: " + request.getServerName());

        Out.println ("Header connection:" + request.getHeader
            ("connection") );

        Out.println ( " Header keep.alive :" + request.getHeader
            ("keep-alive " ) );

        Out.println (" adresse IP du client : adresse IP du client : "
            + request.getRemoteAddr( ) ) ; }

}
```



Problème

Voir correction Problème 2017-2018.



Contact WhatsApp : +237 658395978

Réaliser Par Joël_Yk